Содержание

Введение 4

1. Основание для разработки и анализ предметной области 6

1.1. Анализ объекта исследования 6

1.2. Анализ субъекта 6

1.3. Постановка задачи 7

1.4. Требования к веб-приложению 8

1.5. Выбор языка программирования и технологий 10

1.6. Выбор инструментария 11

2. Анализ требований и определение спецификаций 12

2.1. Диаграмма прецедентов 12

2.2. Описательная спецификация прецедентов 14

2.3. Диаграмма деятельности 15

2.4. Диаграмма классов 18

2.5. Диаграмма последовательности 20

2.6. Диаграмма состояний 23

3. Проектирование программного продукта 26

3.1. Архитектура веб-приложения 26

3.2. Диаграмма пакетов 27

3.3. Диаграмма размещения 28

3.4. Проектирование пользовательского интерфейса (UI/UX) 29

4. Разработка базы данных и API 32

4.1. ER-диаграмма базы данных 32

4.2. Интерфейс пользователя 34

4.3. Разработка клиентской части сайта 37

4.4. Разработка серверной части и администрирование данных 38

5. Тестирование 41

5.1. Модульное тестирование 41

6. Экономическое обоснование 48

6.1. Расчёт затратной части проекта 48

6.2. Расчёт экономии и выгод проекта 48

6.3. Расчёт срока окупаемости проекта 50

6.4. Эффективность внедрения продукта 50

Заключение 52

Список использованных источников 54

Список сокращений 56

Приложение 1 57

Приложение 4 60

Приложение 5 61

Приложение 6 62

Приложение 7 63

Приложение 8 64

Приложение 9 65

Приложение 10 82

Приложение 11 85

Приложение 12 90

Введение

Современное образование активно трансформируется под влиянием цифровых технологий, что приводит к значительному объему данных об успеваемости студентов. Эффективное управление этими данными становится критически важным для оценки качества обучения и индивидуального прогресса. Традиционные методы учёта, часто ручные или децентрализованные, не обеспечивают оперативного доступа к информации и затрудняют глубокий анализ, препятствуя своевременному выявлению проблемных зон.

Возникает острая потребность в специализированных информационных системах, автоматизирующих сбор, хранение, обработку и анализ данных успеваемости. Веб-приложения являются оптимальным решением благодаря их доступности с любого устройства и централизованному управлению данными. Разработка такого приложения позволит оптимизировать работу административного и преподавательского состава, а студентам предоставит возможность оперативно отслеживать собственный прогресс.

Актуальность темы исследования заключается в высокой востребованности современных, централизованных инструментов для управления успеваемостью в условиях цифровизации образования, что напрямую повышает качество обучения и эффективность работы учреждений.

Цель работы: Разработка функционального веб-приложения для автоматизированного сбора, надежного хранения, эффективного анализа и наглядной визуализации данных об успеваемости студентов для обеспечения оперативной информации, и поддержки принятия управленческих решений.

Для достижения поставленной цели в рамках дипломной работы были поставлены следующие задачи:

1. Анализ предметной области и выявление требований.
2. Выбор оптимального технологического стека (C#, JavaScript, HTML, CSS, ASP.NET Core, React.js, SQL Server).

3. Разработка архитектуры приложения и проектирование структуры базы данных.
4. Проектирование интуитивно понятного пользовательского интерфейса.
5. Реализация серверной части (Backend).
6. Разработка клиентской части (Frontend).
7. Интеграция частей приложения и реализация функционала анализа/визуализации.
8. Комплексное тестирование веб-приложения.

Объектом исследования является процесс сбора, обработки и анализа данных об успеваемости студентов в образовательном учреждении. Предметом исследования выступает разрабатываемое веб-приложение как программное средство, автоматизирующее и совершенствующее этот процесс.

Научная новизна может заключаться в адаптации методов анализа данных к образовательной специфике и их реализации в веб-среде, или в оптимизированной архитектуре приложения. Практическая значимость работы состоит в создании готового к внедрению веб-приложения, способного автоматизировать рутинные операции, повысить прозрачность и оперативность анализа данных для принятия решений в образовательных учреждениях.

Структура дипломной работы: Традиционно состоит из введения, шести глав, заключения, списка использованных источников и приложений. Главы охватывают анализ предметной области, требования, проектирование, реализацию, тестирование, экономическое обоснование.

1. Основание для разработки и анализ предметной области

1.1. Анализ объекта исследования

Анализ предметной области включает изучение текущих процессов учета и анализа успеваемости студентов, выявляя проблемы и определяя ключевые требования к будущей системе.

Традиционные подходы к управлению успеваемостью часто характеризуются:

Преобладанием ручных операций: Что ведет к временным затратам, риску ошибок и трудностям консолидации данных.

Децентрализованным хранением данных: Информация разрознена, что затрудняет получение полной картины и замедляет принятие решений.

Ограниченными возможностями анализа: Существующие инструменты не позволяют проводить глубокий анализ успеваемости.

Недостаточной прозрачностью: Студенты и родители часто не имеют оперативного доступа к актуальной информации.

Анализ рынка показал, что комплексные решения часто дороги или недостаточно гибки для специфического глубокого анализа успеваемости. Таким образом, разработка специализированного веб-приложения для централизации данных, автоматизации операций и предоставления аналитического инструментария является актуальной и целесообразной.

1.2. Анализ субъекта

Анализ заинтересованных сторон (субъектов) позволяет определить их роли, потребности и цели для формирования функциональных и нефункциональных требований. Были идентифицированы следующие основные группы субъектов:

- Студенты нуждаются в оперативном и удобном доступе к собственной успеваемости и прогрессу.

- Преподаватели требуют удобного интерфейса для ввода/редактирования оценок, базовой аналитики по своим группам и разграничения доступа к данным.
- Администрации учебного заведения необходимы мощные инструменты анализа и отчетности для принятия управленческих решений, оценки эффективности программ и выявления тенденций.

Понимание потребностей каждой группы послужило основой для детализации требований и создания диаграммы прецедентов.

1.3. Постановка задачи

Эффективный учет и анализ успеваемости студентов критически важен, однако текущие процессы во многих учреждениях не автоматизированы, что приводит к следующим проблемам:

- Трудоемкость: Ручной ввод и отчетность отнимают много времени.
- Раздробленность: Данные хранятся в разных форматах, затрудняя консолидацию.
- Неоперативный доступ: Отсутствует своевременный доступ к актуальной информации для всех участников.
- Ограниченный анализ: Невозможно проводить глубокий анализ и прогнозировать риски.

Для решения этих проблем ставится задача разработать веб-приложение, которое обеспечит:

- Централизованный сбор и структурированное хранение данных успеваемости.
- Безопасный доступ и разграничение прав.
- Удобный интерфейс для ввода, просмотра и редактирования данных.
- Функционал для статистического анализа и анализа динамики.
- Наглядное представление результатов анализа (графики, диаграммы, отчеты).
- Повышение прозрачности оценивания для студентов.

- Снижение трудозатрат персонала. Приложение должно быть масштабируемым, надежным и производительным.

1.4. Требования к веб-приложению

Разрабатываемое веб-приложение является ключевым инструментом для автоматизации процесса работы с успеваемостью студентов. Для эффективного выполнения своих функций оно должно соответствовать следующим требованиям.

1.4.1. Функциональные требования

- Управление пользователями: Регистрация, редактирование, удаление пользователей и управление их ролями (администраторы, преподаватели, студенты); аутентификация и авторизация.
- Управление учебными данными: Добавление/редактирование/удаление групп, курсов, дисциплин, семестров/учебных годов; назначение преподавателей на дисциплины/группы; привязка студентов к группам.
- Учет успеваемости: Ввод/редактирование оценок преподавателями; просмотр списка студентов для ввода оценок.
- Просмотр успеваемости: Студент – свои оценки; Преподаватель – успеваемость своих групп; Администратор – успеваемость всех.
- Анализ успеваемости: Расчет среднего балла (студента/группы, по дисциплине/семестру); анализ динамики; выявление студентов группы риска; сравнение групп.
- Визуализация и отчетность: Представление данных в таблицах, графиках/диаграммах; формирование и экспорт отчетов (ведомости, сводные отчеты).

1.4.2. UX/UI требования

- Интуитивно понятный и адаптивный интерфейс: Логичный дизайн, корректное отображение на разных устройствах (ПК, планшеты, смартфоны).

- Визуальная привлекательность: Современный и приятный для глаз дизайн.
- Эргономичность ввода данных: Быстрый и удобный ввод оценок для преподавателей.
- Наглядность представления информации: Легко читаемые и интерпретируемые данные, понятные графики.
- Обратная связь: Четкое информирование пользователя о действиях (успех, ошибки).

1.4.3. Требования к безопасности

- Аутентификация и авторизация: Надежная проверка подлинности и строгое разграничение прав доступа по ролям.
- Защита данных при передаче: Использование HTTPS.
- Защита от веб-уязвимостей: Меры против SQL-инъекций, XSS, CSRF.
- Резервное копирование данных: Регулярное создание резервных копий БД.
- Журналирование событий: Ведение логов для отслеживания подозрительной активности.

1.4.4. Основные требования к устройствам

- Совместимость с веб-браузерами: Корректная работа в современных версиях популярных браузеров.
- Адаптивность: Автоматическая подстройка интерфейса под разные размеры экранов.
- Производительность устройства: Стандартные требования для запуска современного веб-браузера.
- Наличие подключения к сети Интернет: Для доступа к веб-приложению.

1.4.5. Выбор парадигмы программирования

Будет применен смешанный подход, комбинирующий:

- Объектно-ориентированное программирование (ООП): Основная парадигма для C# и ASP.NET Core на серверной части, обеспечивающая модульность и гибкость.
- Функциональное программирование (ФП): Элементы ФП в JavaScript и React.js на клиентской части для улучшения читаемости и предсказуемости.
- Компонентно-ориентированное программирование: Центральная парадигмы для React.js, позволяющая строить UI из независимых, переиспользуемых компонентов. Такой выбор оптимален для создания современного, поддерживаемого и масштабируемого веб-приложения.

1.5. Выбор языка программирования и технологий

Выбор стека технологий основан на поставленной задаче, требованиях к приложению и пожеланиях, обеспечивая архитектуру, производительность и масштабируемость.

- Серверная часть (Backend): C# и ASP.NET Core.
  - Обоснование: C# — современный ООП язык с развитой .NET экосистемой. ASP.NET Core — высокопроизводительный, кроссплатформенный фреймворк для Web API, поддерживающий MVC. Обширная документация, большое сообщество и инструменты (Visual Studio) — значимые преимущества.
- Клиентская часть (Frontend): HTML, CSS и JavaScript с React.js.
  - Обоснование: HTML и CSS — фундаментальные технологии для веб-интерфейса. JavaScript — стандарт для интерактивности. React.js — эффективная библиотека для создания сложных UI с компонентным подходом, что упрощает разработку, переиспользуемость и поддержку.
- Система управления базами данных (СУБД): Microsoft SQL Server.
  - Обобоснование: Мощная и надежная реляционная СУБД, хорошо интегрирующаяся с .NET технологиями (через Entity Framework

Core). Предоставляет возможности для целостности и безопасности данных, а также масштабирования и оптимизации производительности. Управление через SQL Server Management Studio (SSMS).

Выбранный стек технологий является современным, хорошо зарекомендовавшим себя и оптимален для реализации всех функций приложения, обеспечивая производительность, надежность и удобство сопровождения.

1.6. Выбор инструментария

Для эффективной разработки используется следующий набор инструментов:

- Интегрированная среда разработки (IDE):
  - Microsoft Visual Studio для C# и ASP.NET Core.
  - Visual Studio Code (или WebStorm) для JavaScript и React.js.
- Система управления версиями: Git (с удаленным репозиторием, например, GitHub) для контроля версий и совместной работы.
- Система управления базами данных: SQL Server Management Studio 2022 для администрирования SQL Server.
- Инструменты для работы с Frontend: Npm (менеджер пакетов) для сборки и зависимостей; встроенные инструменты разработчика браузеров (например, Chrome DevTools) для отладки.
- Инструменты для моделирования: Enterprise Architect, draw.io, Lucidchart или PlantUML для UML-диаграмм; инструменты в SSMS или сторонние для ERD.

Выбранный инструментарий обеспечивает полный цикл разработки веб-приложения.

2. Анализ требований и определение спецификаций

2.1. Диаграмма прецедентов

Диаграмма прецедентов (диаграмма вариантов использования) является одним из первых шагов в моделировании функциональных требований системы. Она определяет границы системы, основных участников взаимодействия с ней (акторов) и функции (прецеденты), которые система должна предоставлять.

В контексте веб-приложения для анализа успеваемости студентов, основными акторами являются:

- Администратор: Управляет системой, учетными записями, учебной структурой, имеет доступ к данным и расширенной аналитике.
- Преподаватель: Вводит/редактирует оценки, просматривает успеваемость своих студентов.
- Студент: Просматривает собственную успеваемость и прогресс.

На диаграмме прецедентов (см. рис. 1, с. 29) представлены основные действия, которые могут выполнять пользователи. Таблица 1 детализирует распределение требований по субъектам и прецедентам.

Таблица 1

№	Прецедент (Use Case)	Описание	Акторы
1	Управление пользователями	Добавление, просмотр, редактирование и удаление учетных записей пользователей; назначение ролей в системе.	Администратор
2	Управление учебной структурой	Добавление, просмотр, редактирование и удаление информации о студенческих группах, учебных дисциплинах и семестрах.	Администратор

Продолжение таблицы 1

3	Назначение преподавателей	Закрепление преподавателей за определенными учебными дисциплинами и/или студенческими группами.	Администратор
4	Ввод/редактирование оценок	Внесение новых оценок студентов по дисциплинам и корректировка ранее введенных оценок.	Преподаватель
5	Просмотр успеваемости студента	Получение и отображение подробной информации о собственных академических результатах по всем дисциплинам за различные периоды.	Студент
6	Просмотр успеваемости группы	Получение и отображение сводной информации об успеваемости всех студентов в выбранной группе.	Преподаватель, Администратор
7	Проведение анализа успеваемости	Запуск различных процедур анализа данных успеваемости (расчет статистики, анализ динамики и т.п.).	Преподаватель, Администратор
8	Просмотр результатов анализа	Отображение результатов проведенного анализа в удобном для восприятия формате (таблицы, графики, диаграммы).	Преподаватель, Администратор
9	Формирование отчетов	Генерация структурированных отчетов по успеваемости на основе заданных параметров с возможностью экспорта.	Администратор
10	Авторизация в системе	Процесс идентификации и проверки подлинности пользователя для получения доступа к функционалу системы.	Все Акторы

Рис. 1. Диаграмма прецедентов для "Системы анализа успеваемости"

Таким образом, диаграмма прецедентов и представленная таблица формализуют функциональные требования к веб-приложению с точки зрения его пользователей и закладывают основу для дальнейшего детализированного проектирования системы.

2.2. Описательная спецификация прецедентов

Детальная спецификация каждого прецедента необходима для полного понимания функциональных требований системы. Каждый прецедент подробно описывается в соответствии со стандартным шаблоном, включающим краткое описание, участвующих акторов, предусловия, основной и альтернативные потоки событий, а также постусловия.

Подробная описательная спецификация всех прецедентов, приведенных в Таблице 1, вынесена в Приложение 9.

2.3. Диаграмма деятельности

Диаграмма деятельности используется для моделирования потоков работ и последовательности действий, выполняемых в системе или в рамках конкретного прецедента. Она наглядно отображает шаги процесса, точки ветвления, параллельное выполнение действий и условия переходов. Диаграммы деятельности особенно полезны для детализации логики работы отдельных функций или целых бизнес-процессов, выявленных на этапе анализа прецедентов.

В контексте веб-приложения для анализа успеваемости студентов, диаграммы деятельности разработаны для моделирования ключевых бизнес-процессов и пользовательских сценариев. Примерами таких процессов являются:

- Процесс авторизации пользователя: Моделирует шаги, выполняемые пользователем и системой при входе в приложение, включая ввод учетных данных, их проверку и предоставление доступа или выдачу сообщения об ошибке.

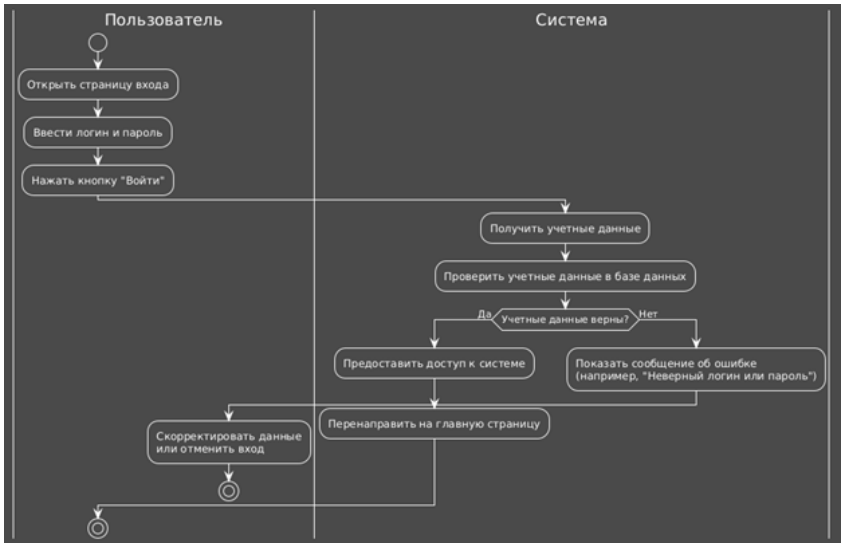

Рис. 2. Диаграмма деятельности для "Процесс авторизации пользователя"

- Процесс ввода и сохранения оценок преподавателем: Показывает последовательность действий преподавателя и системы, начиная от выбора учебного периода, дисциплины и группы, через ввод оценок и валидацию данных, до сохранения информации в базе данных. Учитываются возможные альтернативные потоки (например, некорректный ввод).

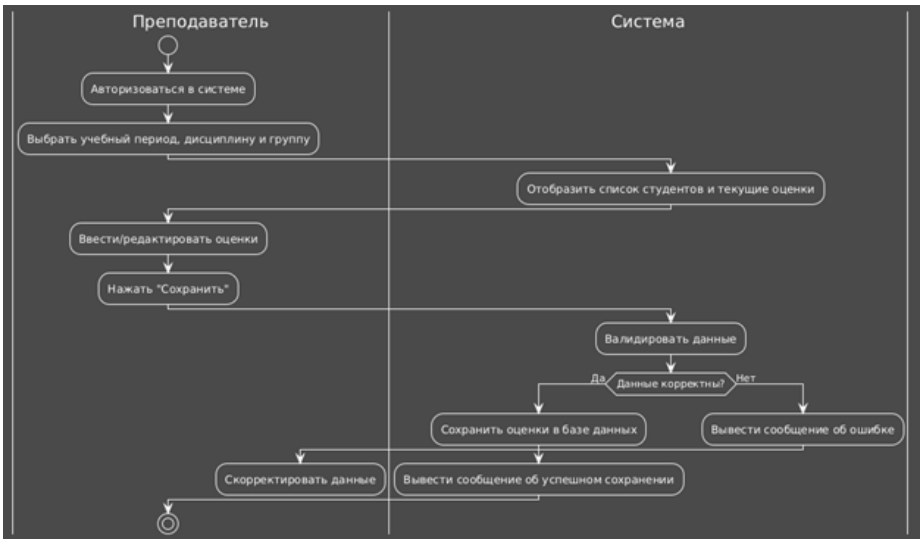

Рис. 3. Диаграмма деятельности для "Процессу ввода и сохранения оценок преподавателем"

- Процесс формирования аналитического отчета: Описывает последовательность действий, связанных с выбором параметров отчета, запуском процесса сбора и анализа данных, формированием структуры отчета и его отображением или экспортом.

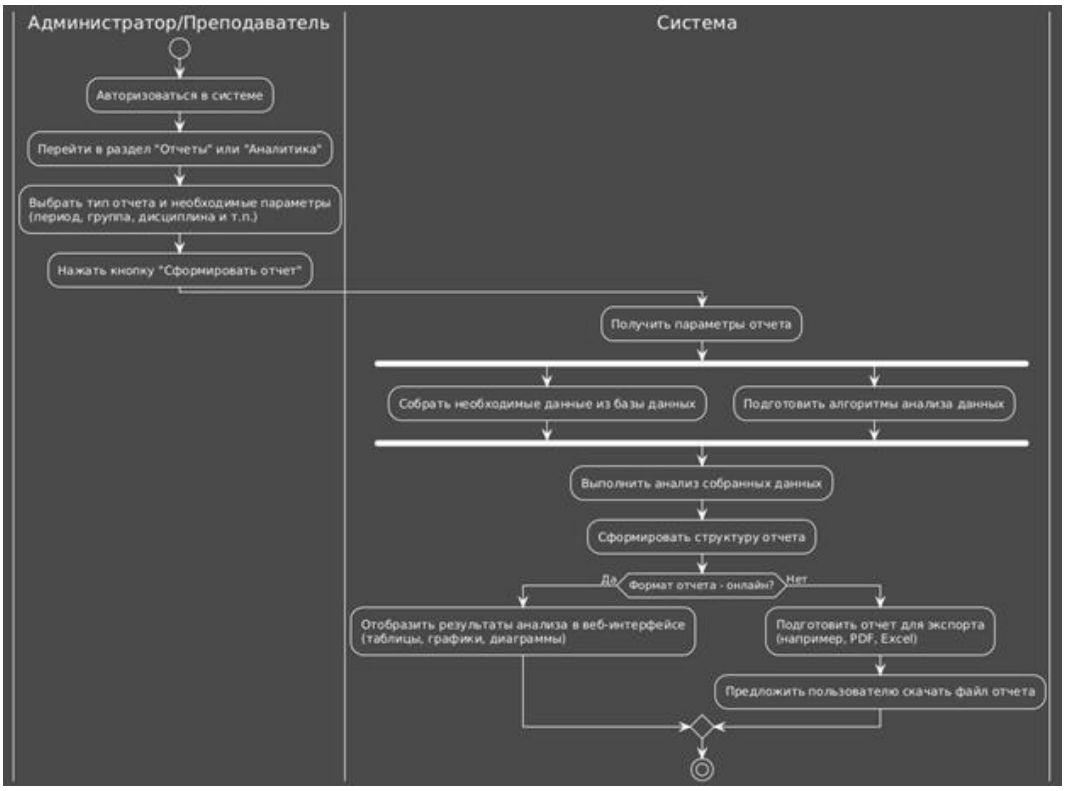

Рис. 4. Диаграмма деятельности для "Процесс формирования аналитического отчета"

- Процесс управления пользователями (например, добавление нового пользователя администратором): Моделирует шаги по вводу данных нового пользователя, выбору его роли, сохранению информации и возможным действиям при дублировании данных или ошибках ввода.

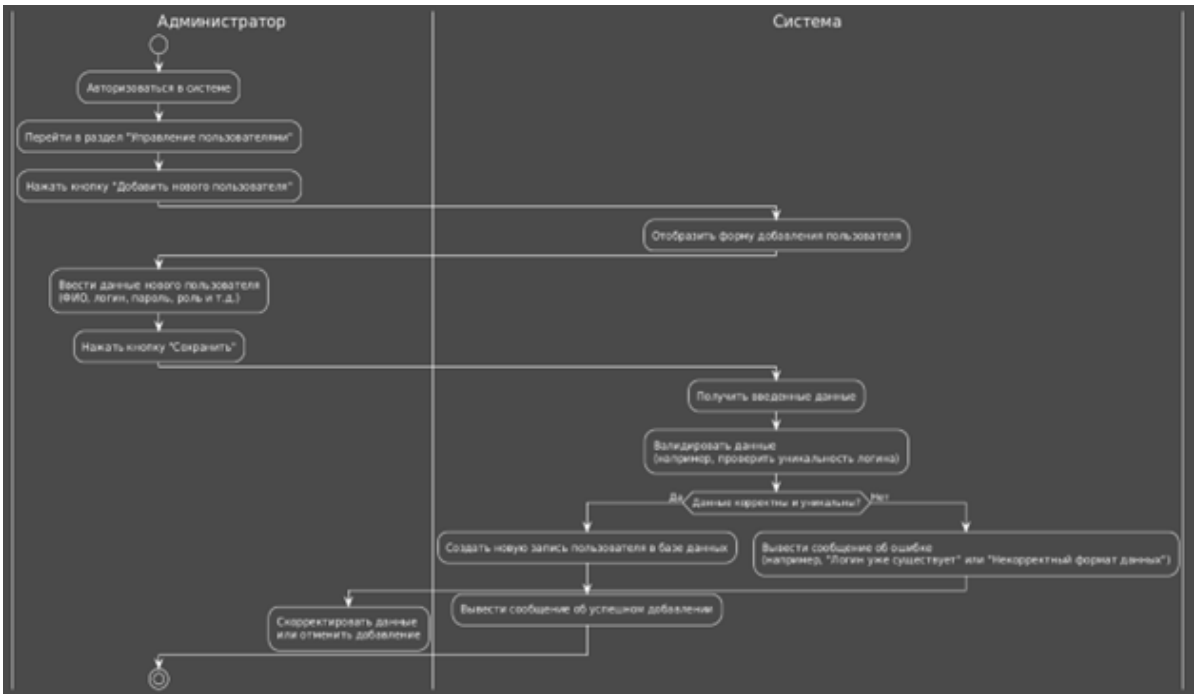

Рис. 5. Диаграмма деятельности для "Процесс управления пользователями (добавление нового пользователя администратором)"

Графическое представление разработанных диаграмм деятельности для описанных ключевых процессов и сценариев использования приведено в Приложении к дипломной работе, наглядно демонстрируя динамическое поведение системы и последовательность выполняемых операций.

2.4. Диаграмма классов

Диаграмма классов представляет собой статическую структурную модель системы. Она определяет типы объектов (классы), их атрибуты (данные), методы (операции) и различные виды отношений между классами (ассоциация, агрегация, композиция, наследование, зависимость). Диаграмма классов является одним из ключевых инструментов объектно-ориентированного анализа и проектирования, служащим основой для дальнейшей реализации программного кода и проектирования структуры базы данных.

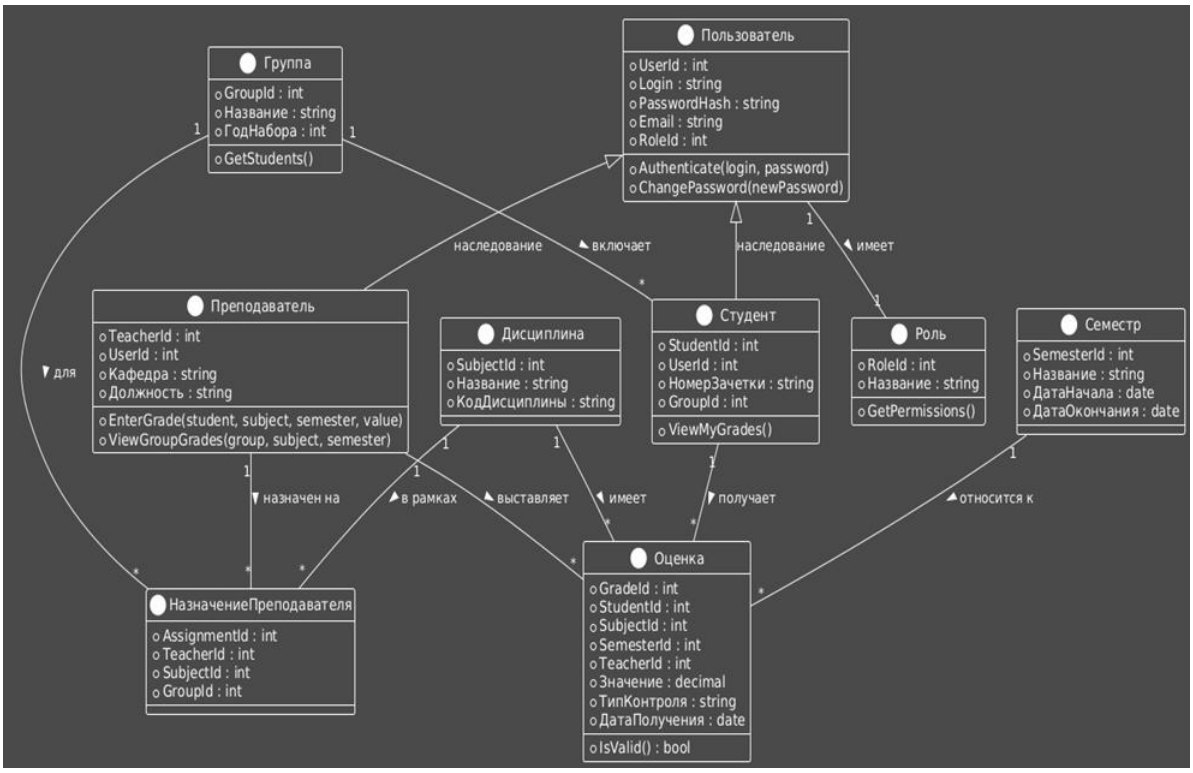

Рис. 6. Диаграмма классов

Диаграмма включает следующие ключевые классы:

- Пользователь (User): Базовый класс для всех акторов (UserId, Login, PasswordHash, Email, RoleId).
- Роль (Role): Определяет уровень доступа (RoleId, Название). Связь "один-ко-многим" с Пользователем.
- Студент (Student): Наследует от Пользователя. Добавляет НомерЗачетки, GroupId.
- Преподаватель (Teacher): Наследует от Пользователя. Включает Кафедра, Должность.
- Группа (Group): Учебная группа (GroupId, Название, ГодНабора). Связь "один-ко-многим" со Студентом.
- Дисциплина (Subject): Учебная дисциплина (SubjectId, Название, КодДисциплины).
- Семестр (Semester): Учебный период (SemesterId, Название, даты начала/окончания).

- Оценка (Grade): Конкретная оценка (GradeId, Значение, ТипКонтроля, ДатаПолучения). Связывает Student, Subject, Semester, Teacher через внешние ключи.
- НазначениеПреподавателя (TeacherSubjectGroup): Моделирует связь "многие-ко-многим" между преподавателями, дисциплинами и группами в заданный период.

Взаимосвязи между классами (обозначенные линиями) показывают, как сущности связаны в рамках предметной области. Детальное понимание структуры классов и их взаимосвязей, представленное на диаграмме, является основой для проектирования структуры базы данных и определения архитектуры программных модулей.

2.5. Диаграмма последовательности

Диаграмма последовательности моделирует взаимодействие между объектами или акторами системы во времени для выполнения определенного сценария использования или операции. Она показывает порядок вызова методов и передачи сообщений, иллюстрируя динамическое поведение системы. Диаграммы последовательности полезны для понимания логики выполнения ключевых операций и выявления потенциальных проблем.

В рамках анализа требований и проектирования будут построены диаграммы последовательности для следующих ключевых сценариев:

- Сценарий "Авторизация пользователя": Иллюстрирует последовательность сообщений между Пользователем, клиентским интерфейсом, серверной частью (API) и базой данных в процессе проверки учетных данных и предоставления доступа.

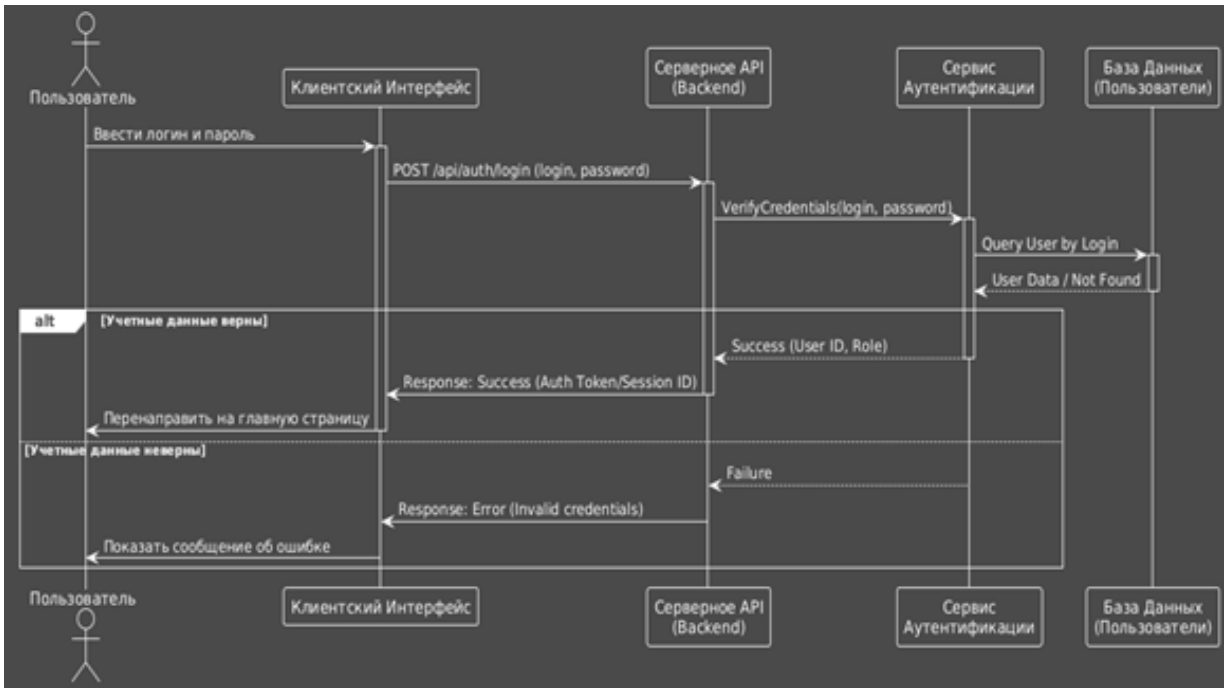

Рис. 7. Диаграмма последовательности для Сценария «Авторизация пользователя»

- Сценарий "Просмотр студентом своей успеваемости": Моделирует взаимодействие между Студентом, клиентом, API и БД при запросе студентом информации о его оценках.

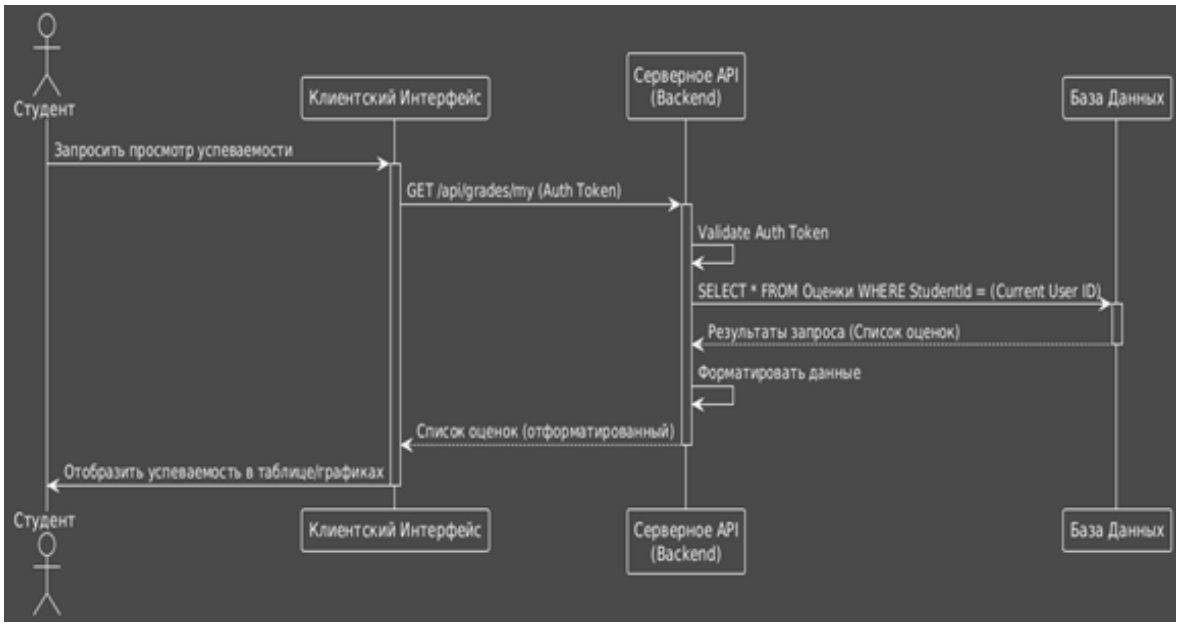

Рис. 8. Диаграмма последовательности для Сценария «Просмотр студентом своей успеваемости»

- Сценарий "Ввод оценки преподавателем": Детализирует шаги взаимодействия при внесении или редактировании оценки преподавателем, включая передачу данных на сервер, валидацию и сохранение в БД.

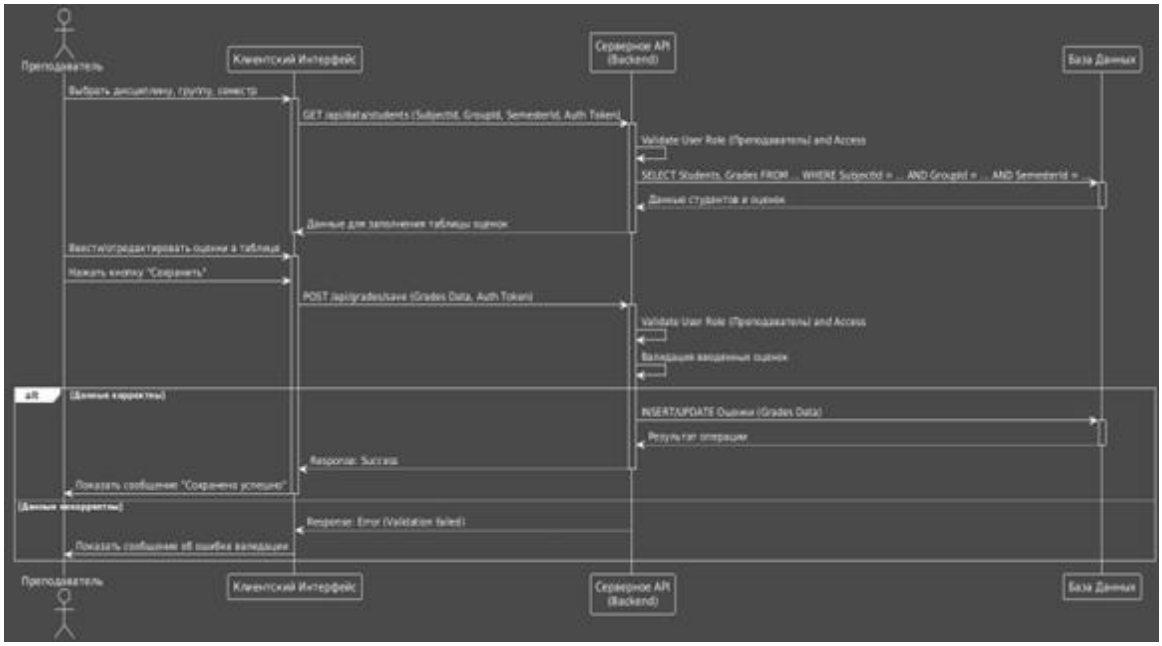

Рис. 9. Диаграмма последовательности для Сценария «Ввод оценки преподавателем»

- Сценарий "Получение аналитических данных": Моделирует взаимодействие при запросе аналитической информации, показывая сбор, обработку и агрегацию данных на сервере и их передачу клиенту для визуализации.

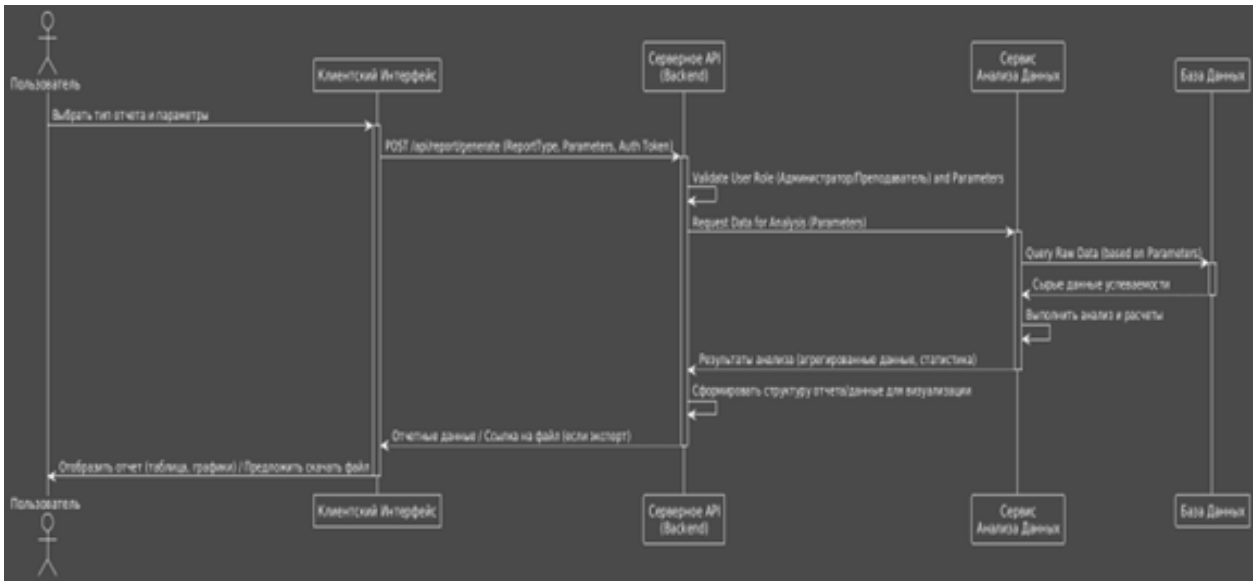

Рис. 10. Диаграмма последовательности для Сценария «Формирование аналитического отчета»

На диаграммах последовательности используются линии жизни объектов, стрелки сообщений, полосы активации и, при необходимости, точки принятия решений или циклы.

2.6. Диаграмма состояний

Диаграмма состояний (диаграмма конечного автомата) моделирует динамическое поведение отдельного объекта или системы, описывая последовательность состояний в ответ на события. Она показывает возможные состояния объекта, переходы между ними и события, вызывающие эти переходы. Диаграммы состояний полезны для объектов со сложным жизненным циклом или зависящих от текущего состояния.

В контексте веб-приложения, диаграммы состояний применимы для моделирования жизненного цикла ключевых сущностей, таких как:

- Объект "Оценка": Оценка может проходить состояния "Черновик", "Выставлена", "Изменена", "Аннулирована" / "Удалена". Переходы вызываются действиями пользователей (например, "Сохранить", "Редактировать").

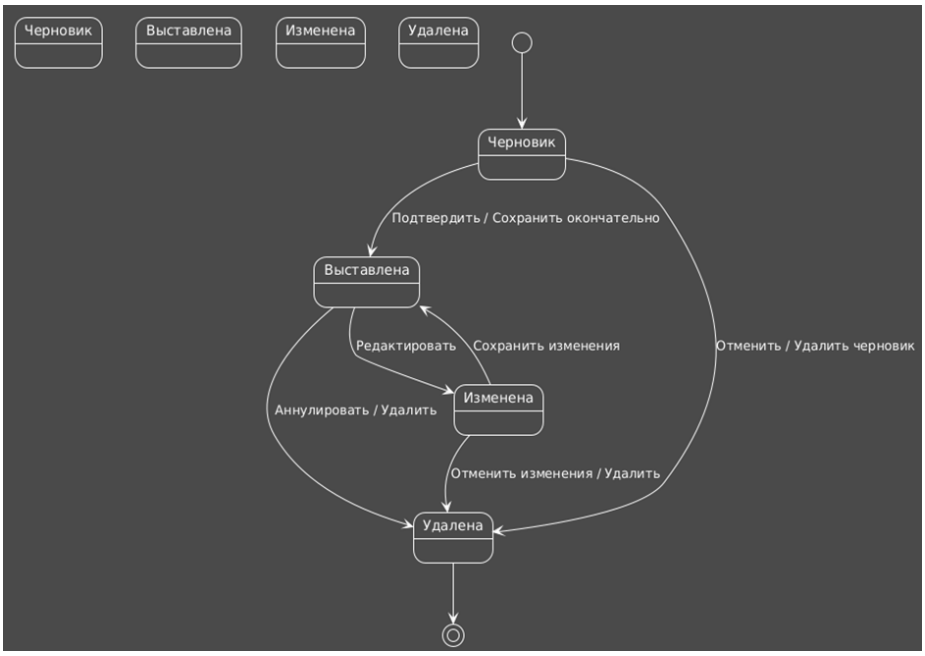

Рис. 11. Диаграмма состояний для объекта «Оценка»

- Объект "Учебный период" (Семестр/Учебный год): Может иметь состояния "Планируется", "Активный", "Завершен", "Архивный". Переходы вызываются датами или действиями администратора.

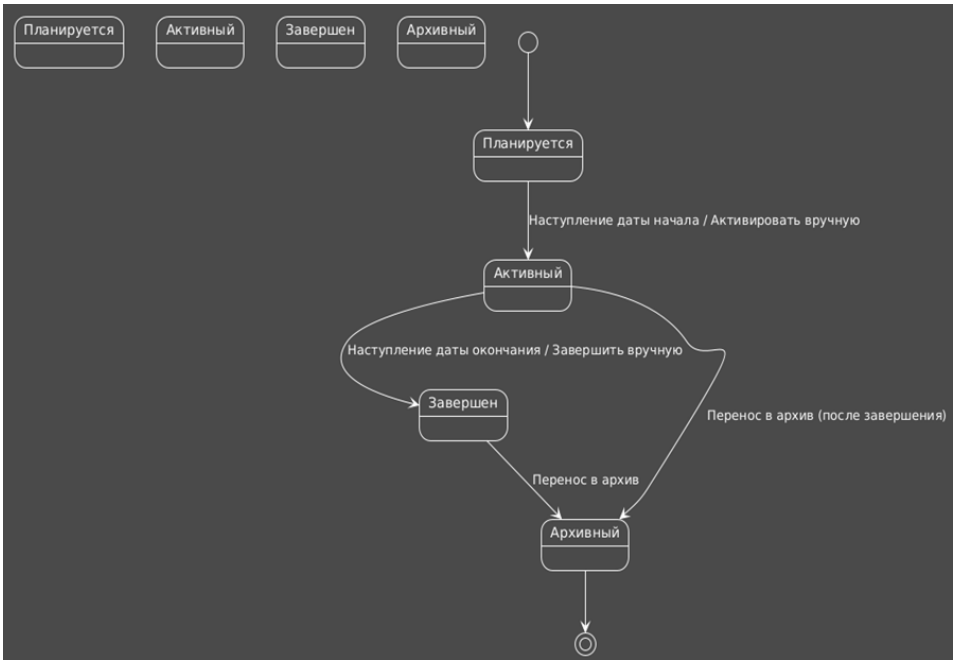

Рис. 12. Диаграмма состояний для объекта «Учебный период» (или Семестр/Учебный год)

- Объект "Учетная запись пользователя": Может иметь состояния "Активна", "Заблокирована", "Удалена". Событиями являются действия администратора (блокировка, разблокировка, удаление) или автоматические процессы.

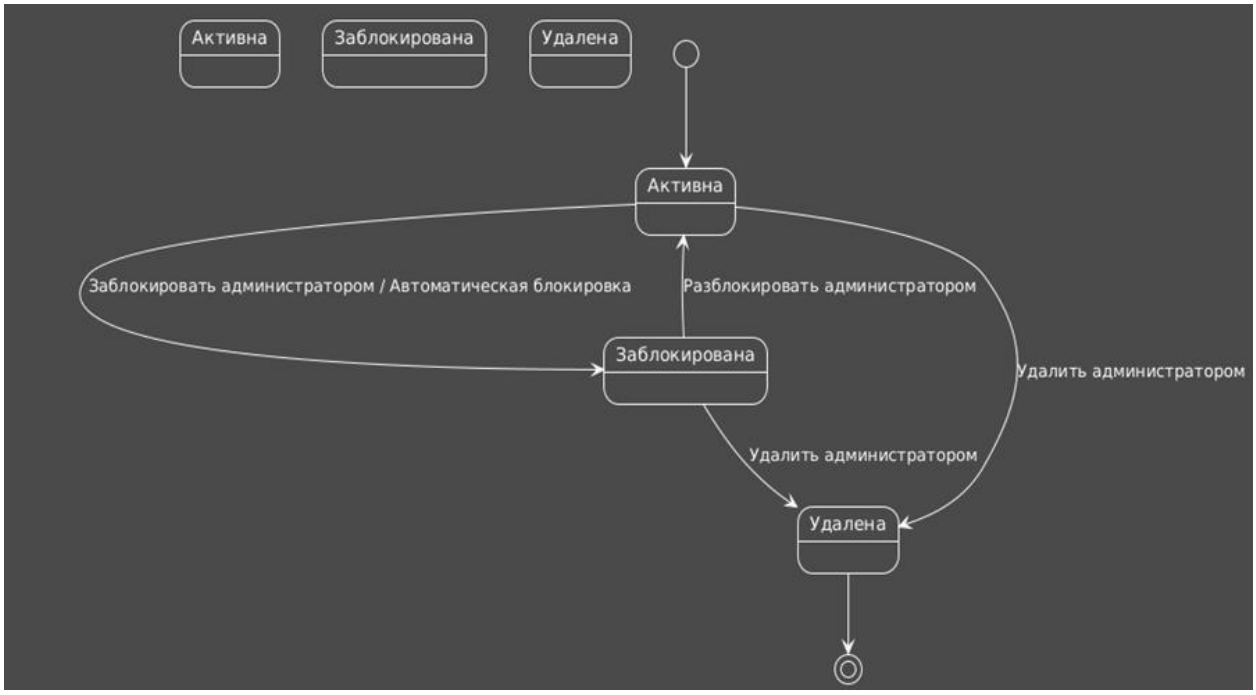

Рис. 13. Диаграмма состояний для объекта «Учетная запись пользователя»

3. Проектирование программного продукта

3.1. Архитектура веб-приложения

Архитектура веб-приложения "Система анализа успеваемости" построена на основе клиент-серверной модели и реализована как одностраничное приложение (SPA). Клиентский интерфейс динамически взаимодействует с серверной частью посредством RESTful API, обмениваясь данными в формате JSON. Это обеспечивает четкое разделение зон ответственности, высокую интерактивность и масштабируемость.

Приложение логически разделено на две основные подсистемы:

1. Подсистема интерфейса пользователя (Клиентская часть): Отвечает за визуализацию и взаимодействие с пользователем.
2. Основная подсистема действий с данными (Серверная часть): Отвечает за бизнес-логику, управление данными, аутентификацию и авторизацию.

В React.js (для клиентской части) применяются принципы паттерна MVVM: Модель (данные приложения из БД), Представление (React-компоненты для отрисовки UI) и ViewModel-подобная логика (управление состоянием компонентов и взаимодействие с данными). Это обеспечивает модульность, реактивное обновление данных, упрощенное создание UI и легкость поддержки.

Рис.14. Общая архитектура веб-приложения

3.1.1. Подсистема интерфейса пользователя

Подсистема интерфейса пользователя (Frontend), разработанная на React.js, предоставляет интуитивно понятный, адаптивный и визуально привлекательный интерфейс для Администратора, Преподавателя и Студента.

Ключевые аспекты подсистемы включают:

- Визуальное представление данных (таблицы, графики, диаграммы).
- Обработка пользовательского ввода через формы.
- Навигация по приложению.
- Динамическое обновление контента без перезагрузки страницы.
- Первичная валидация данных на стороне клиента.
- Эффективное управление состоянием UI.

3.1.2. Основная подсистема действий с данными

Основная подсистема действий с данными (Backend), реализованная на ASP.NET Core (C#), является ядром приложения, предоставляя RESTful API-эндпоинты для клиентского интерфейса.

Ключевые задачи подсистемы включают:

- Аутентификация и авторизация пользователей с разграничением прав доступа.
- Обработка CRUD-операций для всех типов данных (пользователи, группы, дисциплины, оценки).
- Реализация бизнес-логики, включая алгоритмы анализа успеваемости.
- Надежное взаимодействие с базой данных Microsoft SQL Server.
- Строгая валидация данных на стороне сервера.
- Журналирование системных событий и обработка ошибок.

3.2. Диаграмма пакетов

Диаграмма пакетов в UML показывает организацию и иерархию модулей в программном проекте, визуализируя высокоуровневые компоненты системы и их зависимости. В данном проекте пакеты отражают логическое

деление исходного кода на бэкенде (ASP.NET Core) и клиентской части (React.js), включая такие слои как контроллеры, сервисы, модели, компоненты UI и утилиты. (см. рис. 15)

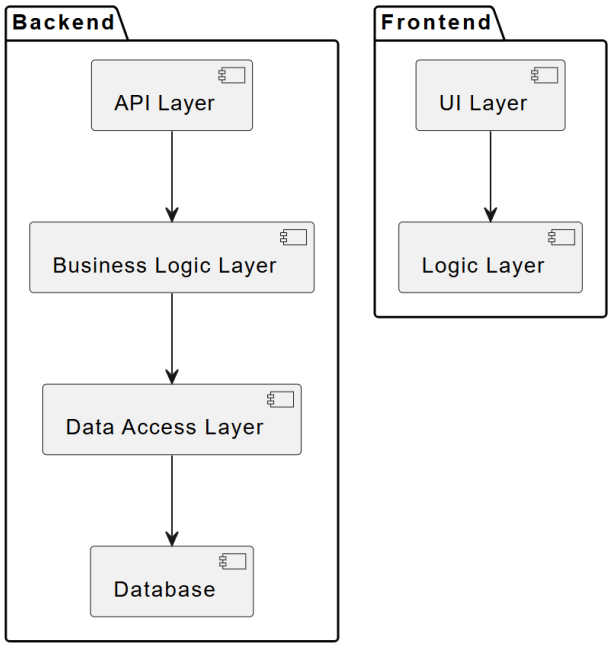

Рис. 15. Диаграмма пакетов.

3.3. Диаграмма размещения

Диаграмма размещения (deployment diagram) иллюстрирует физическое развертывание компонентов программной системы на различных узлах. (см.рис.16)

В контексте веб-приложения "Система анализа успеваемости" основные узлы развертывания и их взаимодействие выглядят следующим образом:

- Клиентский ПК / Мобильное Устройство: Находится на стороне пользователя.
- Клиентский Браузер: Исполняет Frontend-часть на стороне пользователя.
- Сервер веб-приложения: Хостит Frontend (React.js) и Backend (ASP.NET Core).
- Сервер баз данных: Хостит Базу Данных (SQL Server).

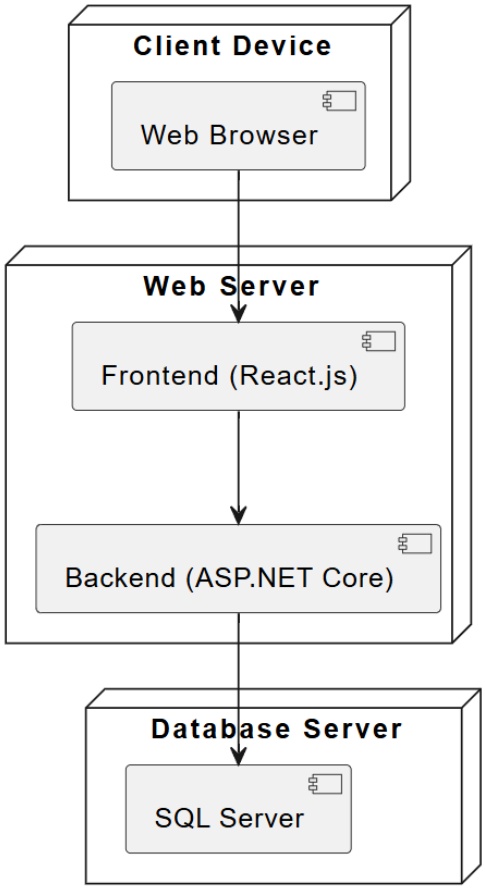

Рис. 16. Диаграмма размещения

Таблица 2

Размещение компонентов в проекте

№	Компонент	Местоположение
1	Клиентский ПК / Мобильное Устройство	На стороне пользователя
2	Клиентский Браузер	На стороне пользователя
3	Фронтенд (React.js)	На сервере веб-приложения
4	Бэкенд (ASP.NET Core)	На сервере веб-приложения
5	База Данных (SQL Server)	На сервере баз данных

3.4. Проектирование пользовательского интерфейса (UI/UX)

Проектирование пользовательского интерфейса (UI) и пользовательского опыта (UX) является ключевым этапом, определяющим

удобство и эффективность взаимодействия с приложением. Основные задачи включают создание интуитивно понятного, адаптивного и эффективного интерфейса, а также наглядное представление информации.

На этапе проектирования UI/UX активно используются Wireframe-диаграммы (каркасные схемы), которые визуализируют основные элементы и их расположение на веб-страницах без детализации стилей. (см. рис. 17, 18) Wireframe-диаграммы были разработаны для ключевых страниц, таких как:

- Страница авторизации и регистрации.
- Главная страница/Дашборд (для каждой роли).
- Страница ввода/редактирования оценок.
- Страница просмотра успеваемости.
- Страница управления пользователями/учебными данными (для администратора).
- Страница отчетов/аналитики.

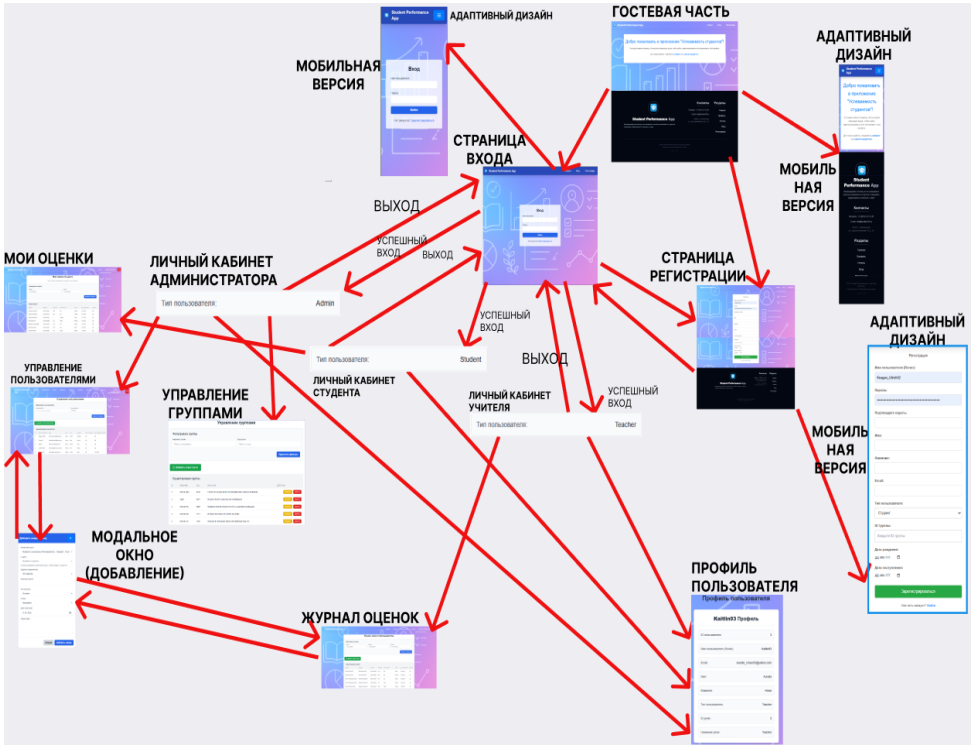

Рис. 17. Диаграмма Wireframe (веб приложения)

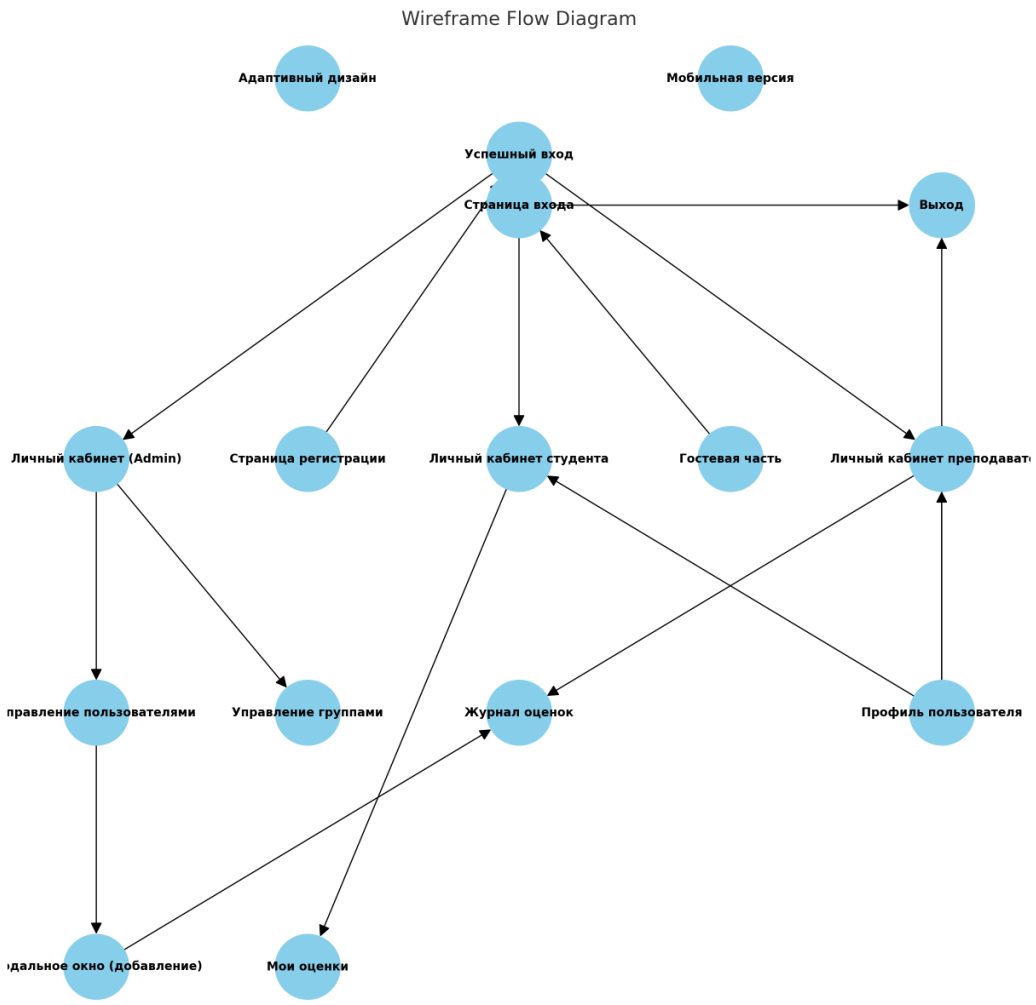

Рис. 18. Диаграмма Wireframe

4. Разработка базы данных и API

4.1. ER-диаграмма базы данных

ER-диаграмма (сущность-связь) – это графический инструмент для моделирования логической структуры базы данных, визуализирующий сущности, их атрибуты и отношения. Она играет ключевую роль в проектировании, обеспечивая четкое понимание взаимосвязей данных. (см. рис. 19)

Структура данных для приложения по анализу успеваемости студентов включает следующие основные сущности:

- Пользователи (Users): Хранит общую информацию о пользователях.
- Студенты (Students): Специфическая информация о студентах, связанная с Пользователями.
- Преподаватели (Teachers): Специфическая информация о преподавателях, связанная с Пользователями.
- Группы (Groups): Информация о студенческих группах.
- Предметы (Subjects): Информация об учебных предметах.
- Семестры (Semesters): Информация об учебных семестрах.

Назначения:

- Преподаватель-предмет-группа (TeacherSubjectGroupAssignments): Связывает преподавателей, предметы, группы и семестры.
- Оценки (Grades): Хранит информацию об оценках студентов, связывая их со студентами и назначениями.

Детальная спецификация атрибутов каждой сущности ER-диаграммы представлена в Приложении 13.

Рис. 19. ER-диаграмма

4.2. Интерфейс пользователя

Пользовательский интерфейс (UI) веб-приложения разработан с акцентом на интуитивность, доступность и адаптивность, обеспечивая комфортное взаимодействие с системой для пользователей с различными уровнями доступа.

4.2.1. Гостевая часть

Гостевая часть приложения доступна без аутентификации и предоставляет основные функции для ознакомления с системой или первого взаимодействия. Ключевые страницы включают:

- Главная страница (/, /home): Точка входа в приложение (см. рис. 18).
- Страница входа (/login): Форма для ввода учетных данных (см. рис. 19).
- Страница регистрации (/register): Позволяет новым пользователям создать учетную запись (см. рис. 20).
- Футер сайта: Нижний колонтитул, доступный на всех страницах (см. рис. 21).

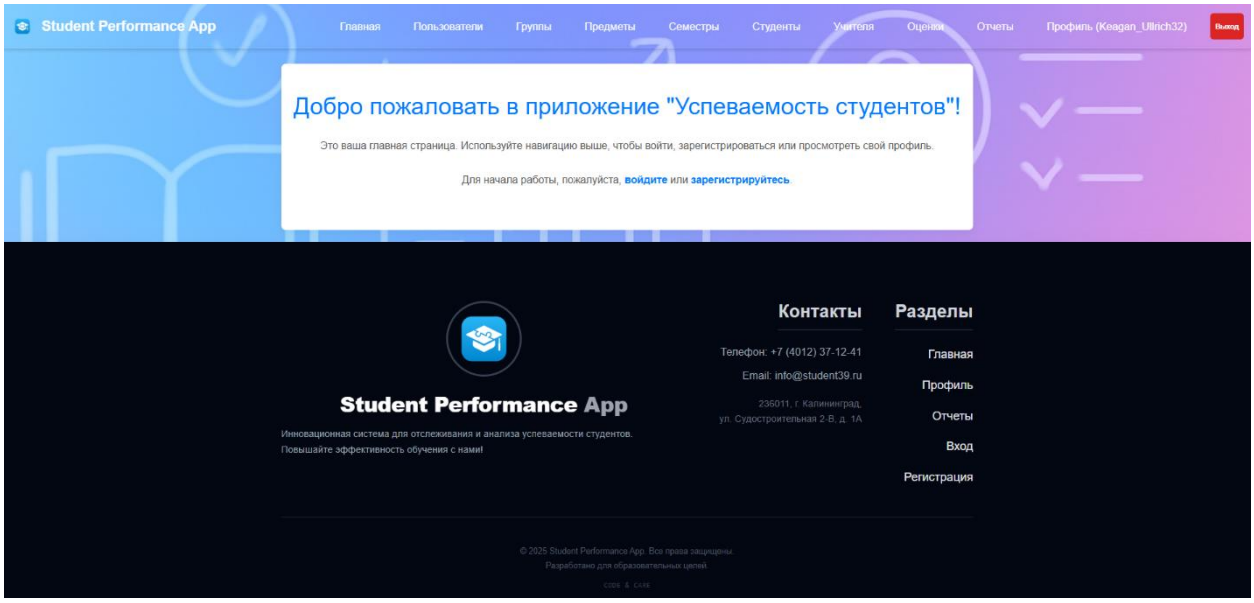

Рис. 20. Главная страница

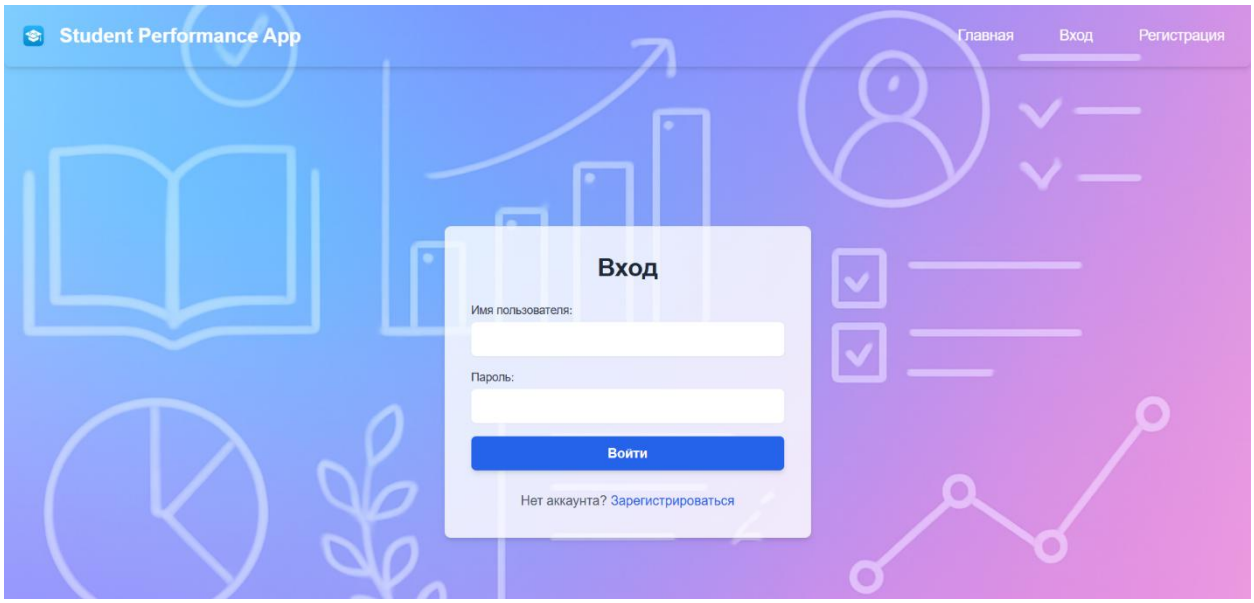

Рис. 21. Страница входа

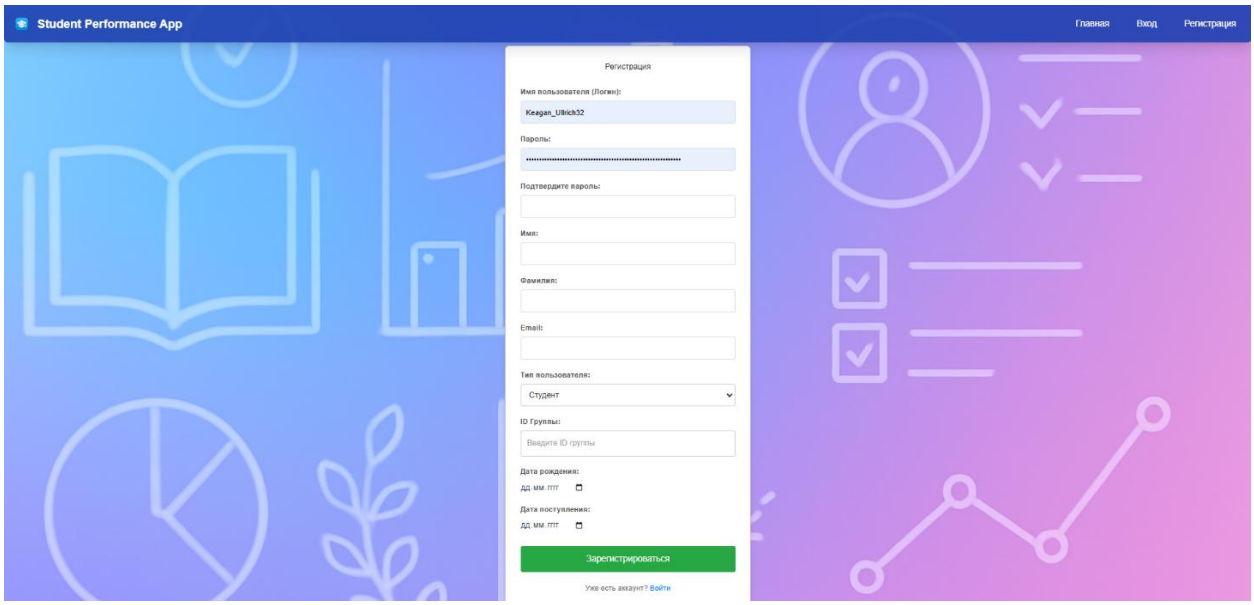

Рис. 22. Страница регистрации

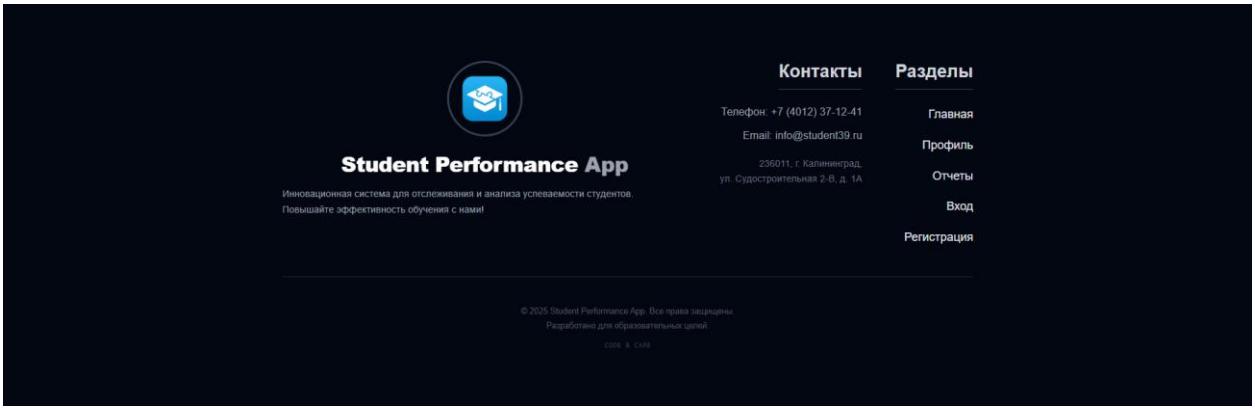

Рис. 23. Футер сайта

Интерфейс гостевой части разработан с использованием Tailwind CSS для современного и адаптивного внешнего вида. Для адаптации сайта под разные устройства создан мобильный интерфейс всех страниц. Графическое представление мобильных версий страниц, включая главную, раздел навигации для администратора, футер, страницы авторизации и регистрации, приведено в Приложении 11.

4.2.2. Личный кабинет пользователя

После успешной аутентификации пользователь получает доступ к личному кабинету, функциональность которого зависит от его роли, обеспечивая персонализированный и безопасный опыт. (см. рис. 24)

- Личный кабинет Администратора (/admin/*): Администраторы имеют полный контроль, включая управление пользователями, студентами, преподавателями, группами, предметами, семестрами, оценками, посещаемостью и заданиями. Все страницы реализованы в виде интерактивных таблиц с функциями фильтрации, поиска, добавления, редактирования и удаления данных, а также с использованием модальных окон.
- Личный кабинет Преподавателя (/teacher/*): Преподаватели имеют доступ к данным, связанным с их учебной деятельностью. Разделы "Журнал оценок", "Посещаемость" и "Задания" позволяют управлять соответствующими записями.
- Личный кабинет Студента (/student/*, /profile): Студенты имеют ограниченный доступ к данным, связанным с их успеваемостью и профилем. Разделы "Мои оценки", "Моя посещаемость", "Мои задания" и "Профиль" предоставляют соответствующую информацию.

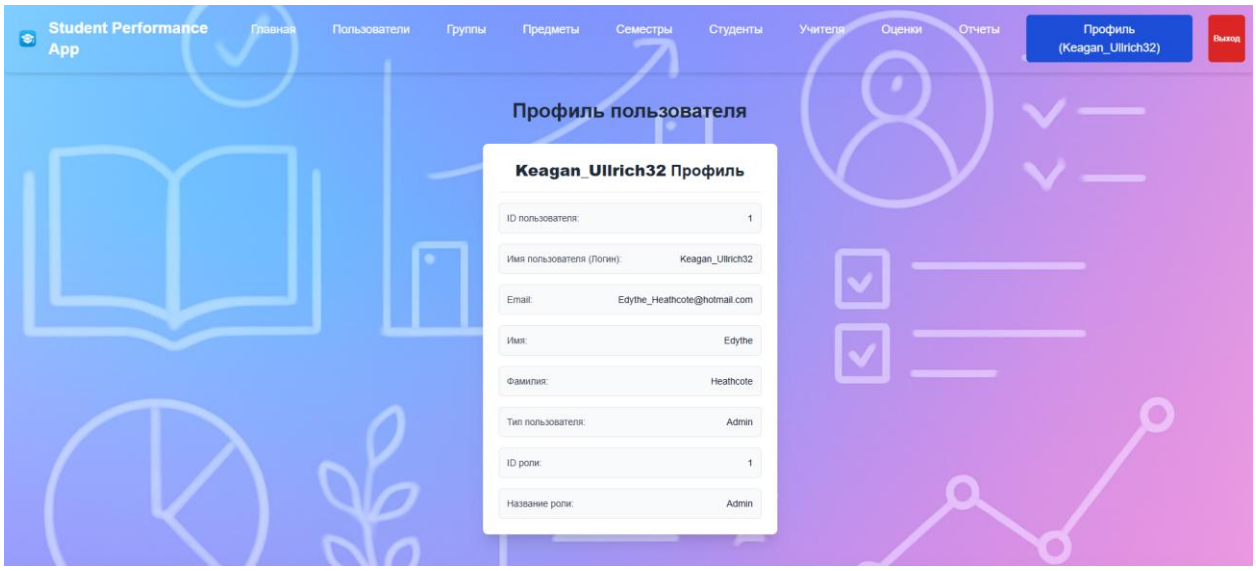

Рис. 24. Личный кабинет пользователя.

Дизайн личного кабинета также выполнен с использованием Tailwind CSS, что обеспечивает единый стиль и адаптивность. Интерфейс таблиц оптимизирован для просмотра больших объемов данных, а модальные окна — для эффективного взаимодействия с формами.

4.3. Разработка клиентской части сайта

Клиентская часть веб-приложения (Frontend) обеспечивает непосредственное взаимодействие пользователя с системой. Основные задачи включали обеспечение легкого доступа к функциям, интуитивно понятной навигации, высокой скорости работы и адаптивного дизайна.

Для реализации клиентской части использовались:

- React.js: Для модульной, компонентной структуры UI.
- JavaScript (ES6+): Для интерактивной логики.
- HTML: Для структуры веб-страниц.
- Tailwind CSS: Для быстрой и гибкой стилизации, упрощения адаптивного дизайна.
- react-router-dom: Для маршрутизации в SPA.
- axios: Для асинхронных HTTP-запросов к API.
- date-fns: Для работы с датами.

- lucide-react: Для современных иконок.

Детальное описание структуры и функциональности клиентских компонентов (Навигация, Защита маршрутов, Вспомогательные UI-компоненты) и подходов к адаптивному дизайну представлено в Приложении 11.

В результате разработки клиентской части создан современный, функциональный и удобный веб-ресурс, соответствующий требованиям к пользовательскому опыту и адаптивности.

4.4. Разработка серверной части и администрирование данных

Администраторская часть веб-приложения является ключевым компонентом, обеспечивающим эффективное управление всеми данными и функциональностью системы, а также разграниченный доступ для преподавателей.

Основные задачи административной части включают:

- Централизованное управление данными: Просмотр, добавление, редактирование и удаление всех сущностей системы.
- Безопасная авторизация и управление доступом: Реализация ролевой модели (RBAC) для разграничения прав.
- Обеспечение целостности и согласованности данных: Валидация входных данных на сервере и обработка ошибок.

Для реализации серверной логики и обеспечения надежности использованы:

- SQL Server: Реляционная СУБД для надежного хранения данных.
- ASP.NET Core (C#): Для разработки мощной и безопасной серверной части (API), обработки запросов и бизнес-логики.

Основные аспекты реализации:

- Настройка базы данных: Развернута база данных SQL Server. Таблицы созданы в соответствии с ER-диаграммой (см. рис. 25). Использовалось

SQL Server Management Studio (SSMS) для управления и взаимодействия.

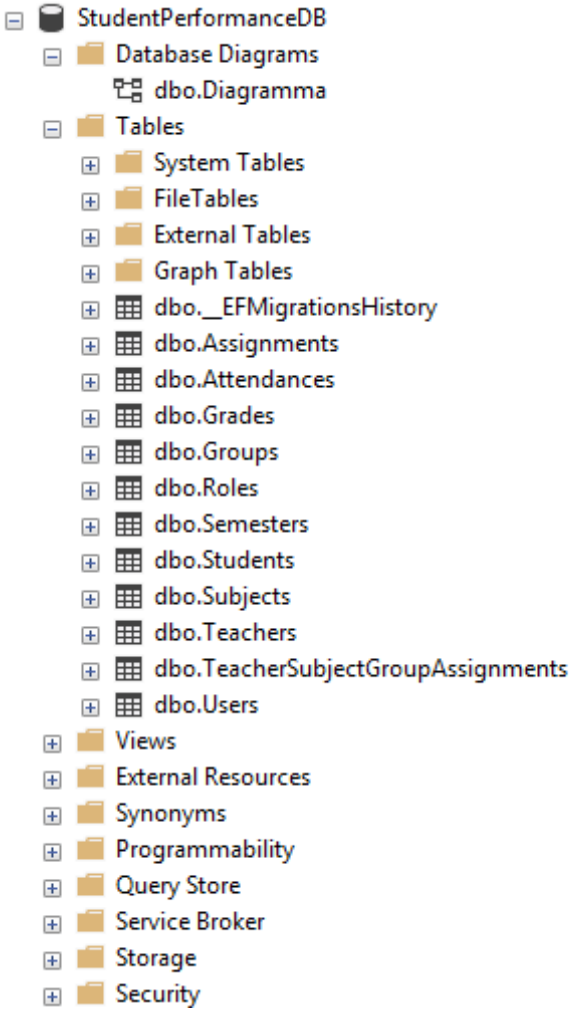

Рис. 25. База данных SQL Server.

- Разработка API-эндпоинтов: Для всех основных сущностей реализованы полные CRUD-операции через RESTful API-эндпоинты, с валидацией входных данных и обработкой ошибок.
- Авторизация и безопасность: Запросы к защищенным эндпоинтам проверяются на наличие JWT-токена. Реализован детальный контроль доступа на основе ролей (RBAC). Для продакшн-среды критически важно использование HTTPS.

- Обработка данных и бизнес-логика: Серверная часть содержит сложную бизнес-логику, включая фильтрацию данных, управление жизненным циклом данных и поддержание взаимосвязей между сущностями.

5. Тестирование

5.1. Модульное тестирование

Модульное тестирование (англ. unit testing) – это процесс проверки отдельных, наименьших логически независимых частей исходного кода программы (модулей или компонентов) на корректность. Этот вид тестирования позволяет оперативно выявить ошибки и гарантировать корректную работу функциональных элементов API приложения для анализа успеваемости студентов.

В рамках данного проекта было выполнено несколько ключевых модульных тестов с использованием интерактивного интерфейса Swagger UI. Swagger UI, генерируемый на основе спецификации OpenAPI вашего API, предоставляет удобный способ для прямого взаимодействия с эндпоинтами, отправки запросов и анализа ответов, подтверждая соответствие поведения API его спецификациям.

5.1.1. Тест успешной регистрации нового пользователя (Студент)

Цель: удостовериться, что API успешно обрабатывает запрос на регистрацию с валидными данными и создает нового пользователя с указанной ролью. (см. рис. 26)

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7242/api/Auth/register' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "newstudentuser",
    "password": "StrongPassword123!",
    "userType": "Student",
    "firstName": "Анна",
    "lastName": "Иванова",
    "email": "anna.ivanova@example.com",
    "groupId": 1,
    "dateOfBirth": "2002-09-15T00:00:00Z",
    "enrollmentDate": "2020-09-01T00:00:00Z"
}'
```

Request URL

```
https://localhost:7242/api/Auth/register
```

Server response

Code	Details
201	Response body

```
{
  "userId": 1044,
  "username": "newstudentuser",
  "firstName": "Анна",
  "lastName": "Иванова",
  "userType": "Student",
  "email": "anna.ivanova@example.com",
  "roleId": 3,
  "roleName": "Student",
  "fullName": "Анна Иванова",
  "groupId": 1,
  "groupName": "Группа 1X21",
  "dateOfBirth": "2002-09-15T00:00:00Z",
  "enrollmentDate": "2020-09-01T00:00:00Z",
  "department": null,
  "position": null,
  "studentId": 1013,
  "teacherId": null
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu,12 Jun 2025 18:21:39 GMT
location: https://localhost:7242/api/Users/1044
server: Kestrel
```

Responses

Code	Description	Links
201	Created	*No links*

Рис. 26. Проверка успешной регистрации пользователя (Студент) в Swagger UI

5.1.2. Тест авторизации пользователя и получения JWT токена

Цель: Удостовериться, что API корректно обрабатывает запрос на вход с валидными учетными данными и возвращает JWT токен для последующей авторизации защищенных эндпоинтов.

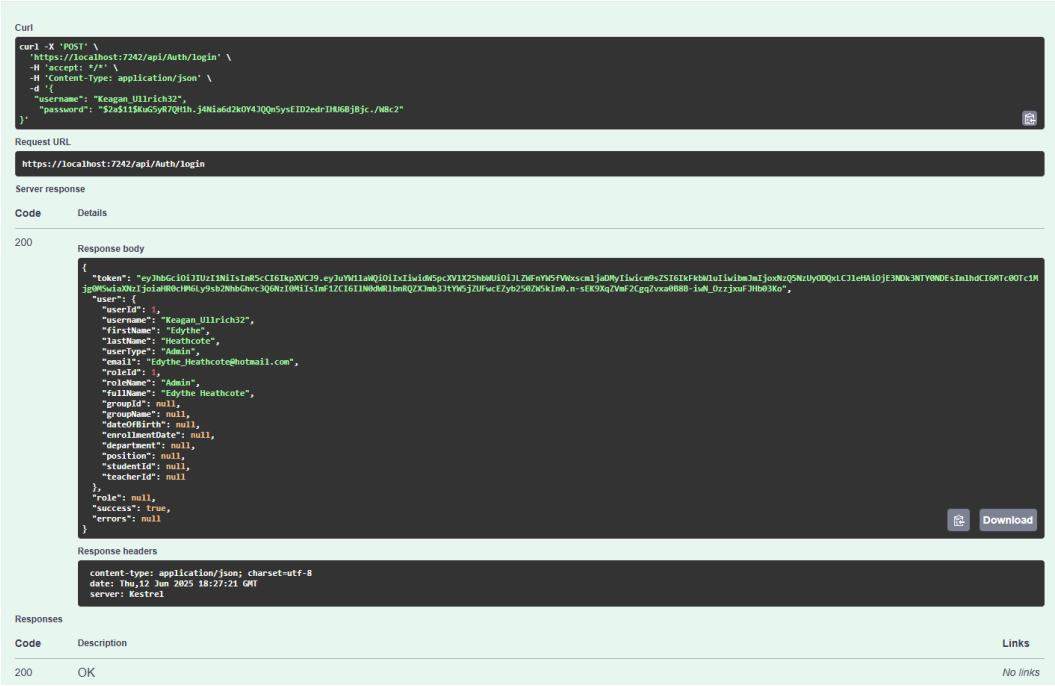

Рис. 27. Проверка авторизации пользователя и получения JWT токена в Swagger UI

5.1.3. Тест регистрации пользователя с существующим именем пользователя

Цель: удостовериться, что API отклоняет запрос на регистрацию, если имя пользователя уже занято, возвращая соответствующую ошибку.

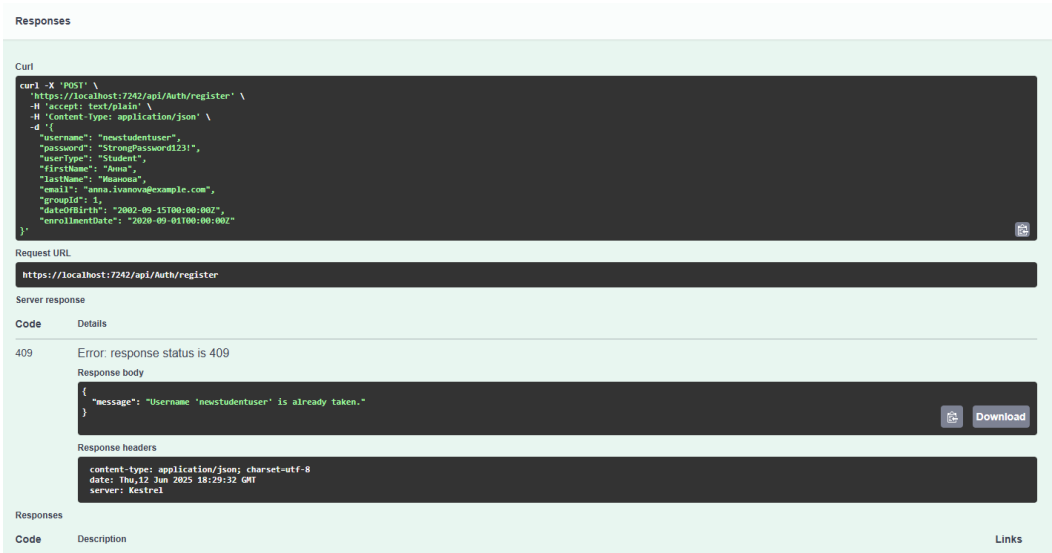

Рис. 28. Проверка регистрации пользователя с существующим именем пользователя в Swagger UI

5.1.4. Тест попытки доступа к списку пользователей без авторизации

Цель: Удостовериться, что API возвращает ошибку несанкционированного доступа (401 Unauthorized) при попытке получить список пользователей без предоставления JWT токена. (см. рис. 29)

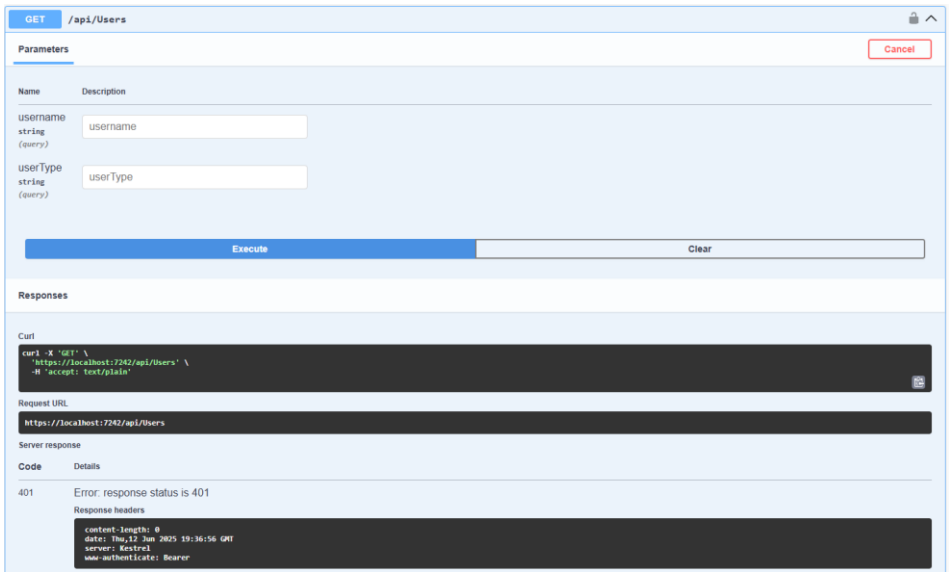

Рис. 29. Проверка попытки доступа к списку пользователей без авторизации в Swagger UI

5.1.5. Тест попытки доступа к административному ресурсу с недостаточными правами

Цель: Удостовериться, что API возвращает ошибку доступа запрещен (403 Forbidden) при попытке пользователя с недостаточными правами (например, Студента или Преподавателя) получить доступ к ресурсу, который доступен только Администраторам. (см. рис. 30)

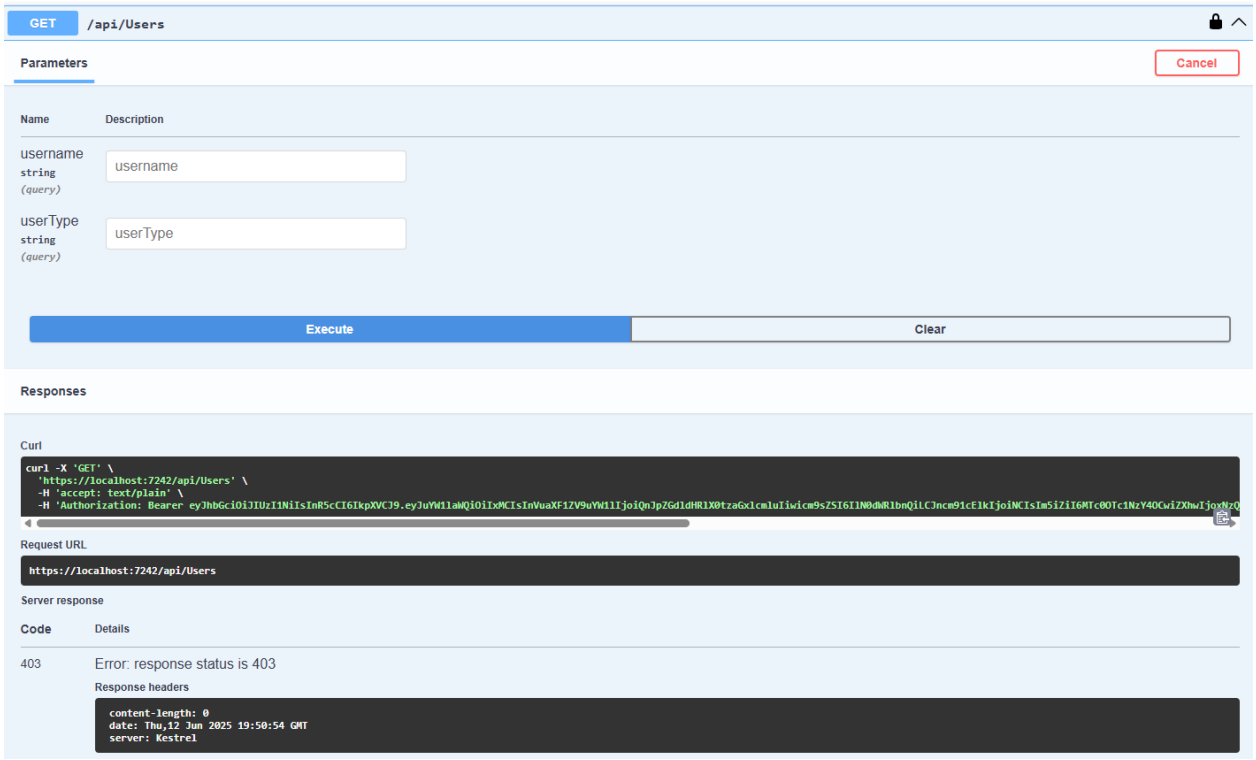

Рис. 30. Проверка попытки доступа к административному ресурсу с недостаточными правами в Swagger UI

5.1.6. Тест получения информации о несуществующей группе

Цель: Удостовериться, что API корректно обрабатывает запрос на получение информации о группе, которая не существует в базе данных, возвращая статус "Не найдено" (404 Not Found).

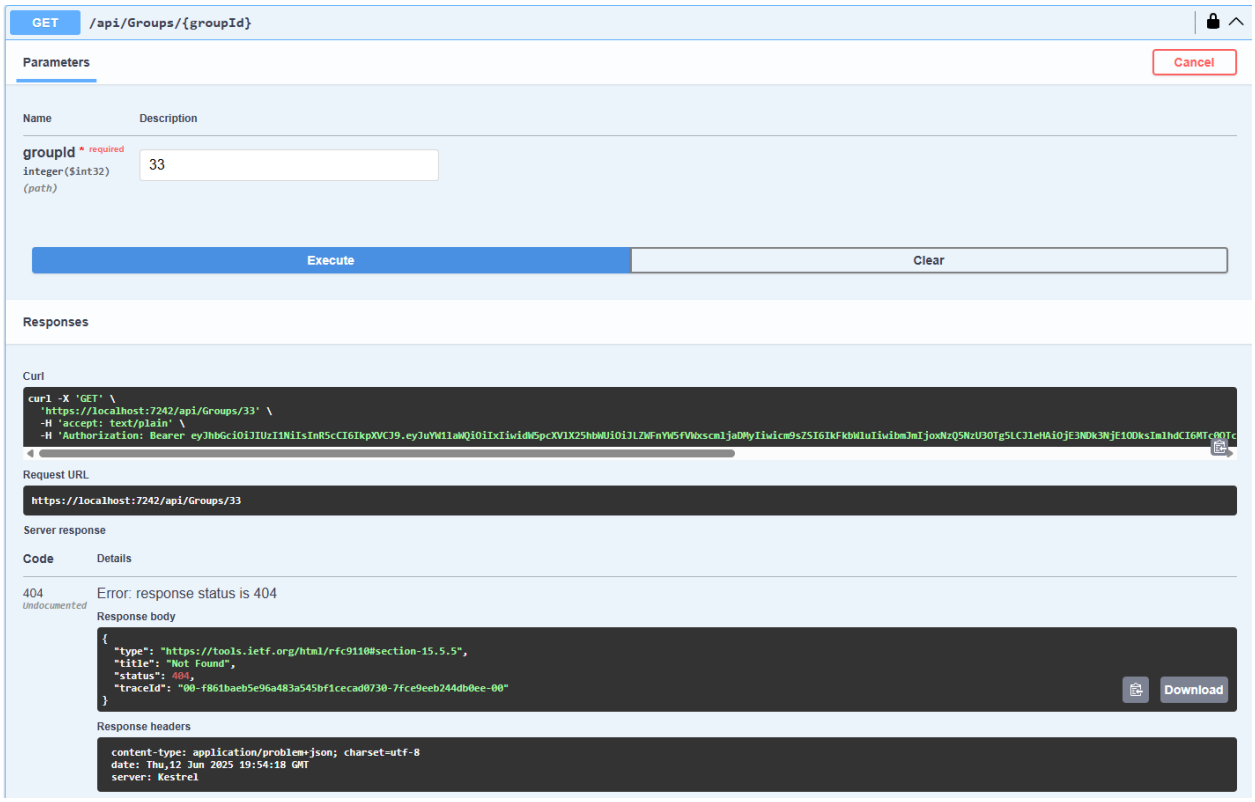

Рис. 31. Проверка получения информации о несуществующей группе в Swagger UI

5.1.7. Тест создания группы с нарушением валидации

Цель: Удостовериться, что API отклоняет запрос на создание группы, если обязательные поля не заполнены или их значения не соответствуют правилам валидации (например, слишком короткое имя), возвращая статус "Плохой запрос" (400 Bad Request).

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7242/api/Groups' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1laWQiOiIxIiwidW5pcXV1X25hbWUiOiJlZWFnYW5fVWxscm1jaDMyIiwicm9sZSI6IkFkbWluIiwibmJmIjoxNzQ5NzU3OTg5LCJleHAiOjE3NDk3NjE1ODksImlhdCI6MTc0OTc
  -H 'Content-Type: application/json' \
  -d '{
  "name": "a",
  "code": "a2",
  "description": "string"
}'
```

Request URL

```
https://localhost:7242/api/Groups
```

Server response

Code	Details
400 *Undocumented*	Error: response status is 400

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Name": [
      "Group name must be between 2 and 100 characters."
    ]
  },
  "traceId": "00-932d15b146038627b2beaa6df171b77d-e4becef56783b7b9-00"
}
```

Download

Response headers

```
content-type: application/problem+json; charset=utf-8
date: Thu,12 Jun 2025 19:55:29 GMT
server: Kestrel
```

Рис. 32. Проверка создания группы с нарушением валидации в Swagger UI

6. Экономическое обоснование

6.1. Расчёт затратной части проекта

Расчёт затратной части проекта включает оценку всех расходов, связанных с разработкой, запуском и потенциальной поддержкой веб-приложения. Поскольку проект является дипломной работой, оценки основываются на гипотетических рыночных ценах для аналогичных услуг.

Единовременные затраты на разработку и внедрение:

- Разработка клиентской части (Frontend - React): 40 000 руб.
- Разработка серверной части и API (Backend - ASP.NET Core, SQL Server): 50 000 руб.
- Разработка базы данных (SQL Server): 15 000 руб.
- Настройка окружения для разработки и тестирования: 10 000 руб.
- Тестирование и отладка: 20 000 руб.
- Написание дипломной работы и документации: 30 000 руб.

Итого единовременные затраты: 165 000 руб.

Ежегодные операционные затраты (гипотетические, для работающего приложения):

- Хостинг сервера/базы данных (виртуальный сервер): 12 000 руб./год
- Доменное имя: 1 000 руб./год
- Потенциальная техническая поддержка и обновления: 60 000 руб./год

Итого ежегодные операционные затраты: 73 000 руб./год

Общая сумма затрат на первый год (единовременные + ежегодные): 165 000 + 73 000 = 238 000 руб.

6.2. Расчёт экономии и выгод проекта

Основная ценность веб-приложения заключается в экономии времени и повышении эффективности для всех участников образовательного процесса. Эти выгоды, хотя и не являются прямым денежным доходом, могут быть конвертированы в эквивалентную экономию.

Средняя оценка ежемесячной экономии времени (гипотетические данные):

- Для преподавателей (на 1 преподавателя):
  - Учёт оценок, формирование отчётов, информирование студентов/родителей: Экономия составляет примерно 6.33 часа/месяц.
  - Примерная ежемесячная экономия: 6.33 часа * 500 руб/час = 3 165 руб.
  - Ежегодная экономия (за 9 учебных месяцев): 28 485 руб.
- Для студентов (на 1 студента):
  - Получение информации об оценках/заданиях, контроль успеваемости: Экономия составляет примерно 0.75 часа/месяц.
  - Примерная ежемесячная экономия: 0.75 часа * 200 руб/час = 150 руб.
  - Ежегодная экономия (за 9 учебных месяцев): 1 350 руб.

Общая ежегодная экономия для учреждения (гипотетически, на 10 преподавателей и 200 студентов):

- Экономия для преподавателей: 28 485 руб. * 10 = 284 850 руб./год
- Экономия для студентов: 1 350 руб. * 200 = 270 000 руб./год Итого общая ежегодная экономия (косвенные выгоды): 284 850 + 270 000 = 554 850 руб./год

Дополнительные, трудноизмеримые выгоды:

- Повышение прозрачности: Мгновенный доступ к информации улучшает взаимопонимание и доверие.
- Снижение ошибок: Автоматизация минимизирует человеческий фактор.
- Улучшенная аналитика: Поддержка принятия управленческих решений.
- Повышение удовлетворенности: Удобство приложения способствует общей удовлетворенности.
- Улучшение репутации учебного заведения: Демонстрация приверженности современным технологиям.

6.3. Расчёт срока окупаемости проекта

Для расчёта срока окупаемости проекта соотносим первоначальные затраты с ежегодной экономией.

Формула расчёта срока окупаемости (Payback Period):

Срок окупаемости (годы)=Ежегодная экономияПервоначальные затраты

Расчёт срока окупаемости:

Срок окупаемости=554,850 руб./год165,000 руб.≈0.297 года

Конвертация в месяцы: 0.297 года×12 месяцев/год≈3.56 месяцев

Таким образом, проект окупится примерно за 3-4 месяца за счет экономии времени и повышения эффективности работы преподавателей и студентов.

6.4. Эффективность внедрения продукта

Внедрение веб-приложения для анализа успеваемости студентов является экономически и операционно эффективным проектом. Несмотря на отсутствие прямых денежных доходов, его ценность проявляется в существенной экономии времени, повышении прозрачности и улучшении качества образовательного процесса.

Эффективность внедрения продукта оценивается по следующим критериям:

- Оптимизация рабочих процессов: Сокращение ручных операций, высвобождающее время персонала.
- Повышение доступности информации: Мгновенный доступ для студентов и родителей, снижение административной нагрузки.
- Улучшение взаимодействия: Централизованная платформа способствует эффективному обмену информацией.
- Поддержка принятия решений: Возможность генерировать аналитические отчёты для улучшения качества образования.
- Современный подход: Повышение имиджа учебного заведения как инновационного.

В конечном итоге, проект является высокоэффективным, поскольку обеспечивает быструю окупаемость инвестиций за счет косвенных выгод и способствует созданию более прозрачной, эффективной и современной образовательной среды.

Заключение

Современное образование сталкивается с вызовами эффективного управления успеваемостью студентов. Традиционные методы учёта и обмена информацией часто ресурсоёмки, непрозрачны и подвержены ошибкам, создавая нагрузку на персонал и замедляя доступ к данным. Актуальность цифровых решений для мониторинга успеваемости возрастает, так как они способны обеспечить централизованный доступ, автоматизировать рутинные операции и повысить общую эффективность образовательного процесса. В рамках данной дипломной работы было разработано веб-приложение для анализа успеваемости студентов, целью которого является оптимизация процесса учёта и предоставления данных об академических достижениях. Приложение эффективно решает обозначенные проблемы благодаря следующим ключевым аспектам:

- Централизованное управление данными: Единая платформа для учёта и хранения информации о студентах, преподавателях, группах, предметах, семестрах, заданиях, оценках и посещаемости, обеспечивающая целостность данных.
- Гибкая система аутентификации и авторизации: Безопасный доступ и разграничение прав пользователей (Администратор, Преподаватель, Студент) реализованы с использованием современных механизмов, таких как JWT.
- Современный и адаптивный пользовательский интерфейс: Клиентская часть, разработанная с использованием React и Tailwind CSS, обеспечивает интуитивно понятный и адаптивный дизайн, позволяя комфортно работать на любых устройствах.
- Надёжная и масштабируемая серверная часть: API приложения реализовано на ASP.NET Core (C#) с использованием SQL Server, что гарантирует высокую производительность, безопасность и возможность дальнейшего масштабирования системы.
- Ключевой функционал для всех участников:

- Студенты: Мгновенный доступ к своим оценкам, посещаемости, информации о заданиях и персональным отчётам.
- Преподаватели: Удобный инструментарий для ввода/редактирования оценок, учёта посещаемости и публикации заданий.
- Администрация: Комплексные аналитические отчёты для оценки общей успеваемости, выявления тенденций и принятия обоснованных управленческих решений.

Разработанное веб-приложение значительно экономит время преподавателей и администрации, повышает удобство для студентов и их родителей, а также способствует повышению прозрачности и общей эффективности образовательного процесса.

Таким образом, внедрение данного веб-приложения предоставляет учебному заведению значительные преимущества, способствуя созданию более современной, прозрачной и эффективной образовательной среды. Эффективное использование цифровых технологий открывает новые возможности для оптимизации учебного процесса и повышения качества образования.

Список использованных источников

1. Федеральный закон от 27.07.2006 №152-ФЗ "О персональных данных" (дата обращения 09.05.2025);
2. ГОСТ 34.602-89 "Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы" (дата обращения 05.05.2025);
3. ГОСТ Р ИСО/МЭК 12207-2010 "Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств" (дата обращения 05.05.2025);
4. Бхаргава А. "Грокаем алгоритмы. Иллюстрированное пособие для программистов и любопытных" – СПб.: "Питер", 2019 (дата обращения 10.05.2025);
5. Бухольц Г. "Безопасность веб-приложений. Практическое руководство" – М.: "ДМК Пресс", 2022 (дата обращения 14.06.2025);
6. Иванов П. А. "ASP.NET Core и Entity Framework Core: Практическое руководство" – М.: "ДМК Пресс", 2023 (дата обращения 08.05.2025);
7. Исаев А. В. "Визуализация данных в веб-приложениях" – М.: "Горячая линия - Телеком", 2021 (дата обращения 14.06.2025);
8. Керриган Б., Ритчи Д. "Язык программирования С" – М.: "Вильямс", 2019 (дата обращения 03.05.2025);
9. Майер Э., Болдуин Л. "CSS. Подробное руководство" – М.: "Символ-Плюс", 2018 (дата обращения 24.05.2025);
10. Мартин Р. "Чистая архитектура. Искусство разработки программного обеспечения" – СПб.: "Питер", 2018 (дата обращения 14.06.2025);
11. Робсон Д. "SQL. Справочник" – М.: "Лори", 2020 (дата обращения 14.06.2025);
12. Ситник Р. М. "Тестирование программного обеспечения. Базовый курс" – М.: "ДМК Пресс", 2019 (дата обращения 07.05.2025);

13. Троелсен Э., Сэлтер Ф. "Язык программирования C# и платформа .NET" – СПб.: "Питер", 2023 (дата обращения 14.06.2025);
14. Флэнаган Д. "JavaScript: Подробное руководство" – СПб.: "Питер", 2020 (дата обращения 10.05.2025);
15. Фримен А. "ASP.NET Core MVC с примерами на C# 8" – СПб.: "Питер", 2021 (дата обращения 08.05.2025);
16. Шаламов В. Д. "Разработка веб-приложений на React: от новичка до профессионала" – М.: "Эксмо", 2023 (дата обращения 10.05.2025);
17. Документация React [Электронный ресурс] https://react.dev/ (дата обращения 10.05.2025);
18. Официальная документация ASP.NET Core [Электронный ресурс] https://learn.microsoft.com/ru-ru/aspnet/core/ (дата обращения 10.05.2025);
19. Официальная документация Microsoft SQL Server [Электронный ресурс] https://learn.microsoft.com/ru-ru/sql/sql-server/ (дата обращения 11.05.2025);
20. Официальная документация OpenAPI Specification (Swagger) [Электронный ресурс] https://swagger.io/specification/ (дата обращения 11.05.2025).

Список сокращений

API – Application Programming Interface (интерфейс программирования приложений)

ASP.NET Core – (открытая кроссплатформенная среда для создания современных веб-приложений и сервисов)

CSS – Cascading Style Sheets (каскадные таблицы стилей)

CRUD – Create, Read, Update, Delete (создание, чтение, обновление, удаление)

DTO – Data Transfer Object (объект передачи данных)

ER – Entity-Relationship (сущность-связь, модель данных)

HTML – HyperText Markup Language (язык гипертекстовой разметки)

HTTPS – HyperText Transfer Protocol Secure (защищенный протокол передачи гипертекста)

IDE – Integrated Development Environment (интегрированная среда разработки)

JSON – JavaScript Object Notation (текстовый формат обмена данными)

JWT – JSON Web Token (открытый стандарт для создания токенов доступа)

ООП – Объектно-ориентированное программирование

ORM – Object-Relational Mapping (объектно-реляционное отображение)

ПК – Персональный компьютер

REST – Representational State Transfer (архитектурный стиль взаимодействия компонентов распределенного приложения)

SQL – Structured Query Language (язык структурированных запросов)

СУБД – Система управления базой данных

UI – User Interface (пользовательский интерфейс)

URL – Uniform Resource Locator (унифицированный указатель ресурса)

UX – User Experience (пользовательский опыт)

Приложение 1

Неполный код компонента авторизации (React)

```
async register(registrationData) {
    console.log("AuthService - Attempting registration with data:", registrationData);
    try {
        const response = await api.post(API_URL + "register", registrationData);
        console.log("AuthService - Registration successful:", response.data);
        return response.data;
    } catch (error) {
        if (error.response) {
            console.error("AuthService - Registration error: Response data:", error.response.data);
            console.error("AuthService - Registration error: Response status:", error.response.status);
            console.error("AuthService - Registration error: Response headers:", error.response.headers);
        } else if (error.request) {
            console.error("AuthService - Registration error: No response received from server for request:", error.request);
        } else {
            console.error("AuthService - Registration error: Error setting up request:", error.message);
        }
        console.error("AuthService - Registration error (full object):", error);

        throw new Error(error.response?.data?.message || error.message || "Произошла ошибка при регистрации. Пожалуйста, попробуйте еще раз.");
    }
}

logout() {
    localStorage.removeItem("user");
    console.log("AuthService - User data removed from localStorage.");
}

getCurrentUser() {
    const userStr = localStorage.getItem("user");
    console.log("AuthService - getCurrentUser: Raw data from localStorage:", userStr);
    try {
        const parsedUser = userStr ? JSON.parse(userStr) : null;
        console.log("AuthService - getCurrentUser: Parsed user object from localStorage:", parsedUser);

        if (parsedUser && (!parsedUser.accessToken || !parsedUser.userType)) {
            console.warn("AuthService - getCurrentUser: User object in localStorage is missing 'accessToken' or 'userType'. Clearing corrupted data.", parsedUser);
            localStorage.removeItem("user");
            return null;
        }
        return parsedUser;
    } catch (e) {
        console.error("AuthService - getCurrentUser: Error parsing user data from localStorage. Clearing corrupted data:", e);
        localStorage.removeItem("user");
        return null;
    }
}
```

Рис. 33. React-код компонента авторизации (фрагмент)

Приложение 2

Неполный код компонента отображения оценок (React)

```
                </button>
            </div>
        </form>
    </div>

    {/* Grades Table */}
    {grades.length === 0 ? (
        <p className="text-center text-gray-600 py-8 bg-blue-50 border border-blue-200 rounded-lg">Нет зарегистрированных оценок или данные не найдены.</p>
    ) : (
        <div className="overflow-x-auto bg-white rounded-lg shadow-md border border-gray-200">
            <table className="min-w-full divide-y divide-gray-200">
                <thead className="bg-gray-100">
                    <tr>
                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[60px]">ID</th>
                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[150px]">Студент</th>
                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[150px]">Преподаватель</th>
                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[150px]">Предмет</th>
                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[100px]">Семестр</th>
                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[150px]">Задание</th>
                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[70px]">Значение</th>
                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[120px]">Тип</th>
                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[100px]">Статус</th>
                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[120px]">Дата</th>
                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[200px]">Примечания</th>
                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[150px]">Действия</th>
                    </tr>
                </thead>
                <tbody className="bg-white divide-y divide-gray-200">
                    {grades.map(grade => (
                        <tr key={grade.gradeId}>
                            <td className="px-3 py-3 whitespace-nowrap text-sm font-medium text-gray-900">{grade.gradeId}</td>
                            <td className="px-3 py-3 text-sm text-gray-900 whitespace-normal">{grade.studentName || 'Н/Д'}</td>
                            <td className="px-3 py-3 text-sm text-gray-900 whitespace-normal">{grade.teacherName || 'Н/Д'}</td>
                            <td className="px-3 py-3 text-sm text-gray-900 whitespace-normal">{grade.subjectName || 'Н/Д'}</td>
                            <td className="px-3 py-3 text-sm text-gray-900 whitespace-normal">{grade.semesterName || 'Н/Д'}</td>
                            <td className="px-3 py-3 text-sm text-gray-900 whitespace-normal">{grade.assignmentTitle || 'Н/Д'}</td>
                            <td className="px-3 py-3 whitespace-nowrap text-sm text-gray-900 min-w-[80px]">{grade.value}</td>
                            <td className="px-3 py-3 whitespace-nowrap text-sm text-gray-900 min-w-[100px]">{grade.controlType}</td>
                            <td className="px-3 py-3 whitespace-nowrap text-sm text-gray-900 min-w-[80px]">{grade.status}</td>
                            <td className="px-3 py-3 whitespace-nowrap text-sm text-gray-900 min-w-[100px]">{new Date(grade.dateReceived).toLocaleDateString()}</td>
                            <td className="px-3 py-3 text-sm text-gray-900 whitespace-normal max-w-[200px] overflow-hidden text-ellipsis min-w-[150px]">
                                {grade.notes || 'Н/Д'}
                            </td>
                            <td className="px-3 py-3 text-right text-sm font-medium whitespace-nowrap min-w-[120px]">
                                {(currentUserRole === 'Admin' || currentUserRole === 'Teacher') ? (
                                    <div className="flex space-x-2 justify-end">
```

Рис. 34. React-код компонента отображения оценок (фрагмент)

Приложение 3

Неполный код компонента отображения отчетов по успеваемости (React)

```
            {/* Report Display Section (Card equivalent) */}
            <div className="rounded-lg shadow-md bg-white border border-gray-200"> {/* Added bg-white and border for clarity */}
                <div className="bg-gray-100 px-6 py-4 rounded-t-lg"> {/* Header for report results */}
                    <h3 className="text-xl font-semibold text-gray-700">Результаты отчета</h3>
                </div>
                <div className="p-0"> {/* Body, p-0 as table handles its own padding */}
                    {reportLoading ? (
                        <div className="text-center py-8">
                            <CustomSpinner size="md" />
                            <span className="ml-3 text-gray-600">Генерация отчета, пожалуйста, подождите...</span>
                        </div>
                    ) : reportData.length === 0 ? (
                        <p className="text-center text-gray-600 py-8 bg-blue-50 border border-blue-200 rounded-lg m-4">Отчет не сформирован или нет данных по выбранным параметрам.</p>
                    ) : (
                        <div className="overflow-x-auto rounded-b-lg"> {/* Adjusted for rounded bottom */}
                            <table className="min-w-full divide-y divide-gray-200">
                                <thead className="bg-gray-100">
                                    <tr>
                                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[150px]">Студент</th>
                                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[150px]">Предмет</th>
                                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[120px]">Средняя оценка</th>
                                        <th className="px-3 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider whitespace-nowrap min-w-[120px]">Количество оценок</th
                                    </tr>
                                </thead>
                                <tbody className="bg-transparent divide-y divide-gray-200">
                                    {reportData.map((item, index) => (
                                        <tr key={index}>
                                            <td className="px-3 py-3 whitespace-nowrap text-sm text-gray-900">{item.studentFirstName} {item.studentLastName}</td>
                                            <td className="px-3 py-3 whitespace-nowrap text-sm text-gray-900">{item.subjectName}</td>
                                            <td className="px-3 py-3 whitespace-nowrap text-sm text-gray-900">{item.averageGrade !== null ? item.averageGrade.toFixed(2) : 'Н/Д'}</td>
                                            <td className="px-3 py-3 whitespace-nowrap text-sm text-gray-900">{item.gradeCount}</td>
                                        </tr>
                                    ))}
                                </tbody>
                            </table>
                        </div>
                    )}
                </div>
            </div>
        </div>
    );
};

export default ReportsPage;
```

Рис. 35. React-код компонента отображения отчетов по успеваемости (фрагмент)

Приложение 4

Неполный код сервиса для взаимодействия с API (React)

```
    baseURL: 'https://localhost:7242',
    headers: {
        'Content-Type': 'application/json',
    },
});

// Добавляем перехватчик запросов (request interceptor)
api.interceptors.request.use(
    (config) => {
        const userStr = localStorage.getItem('user');
        let user = null;
        try {
            user = userStr ? JSON.parse(userStr) : null;
        } catch (e) {
            console.error("api.js - Error parsing user from localStorage:", e);
            localStorage.removeItem('user');
        }

        if (user && user.accessToken) {
            config.headers.Authorization = `Bearer ${user.accessToken}`;
            console.log('api.js - JWT token added to request headers.');
        } else {
            console.log('api.js - No valid JWT token found in local storage or user object is corrupted.');
        }
        return config;
    },
    (error) => {
        console.error('api.js - Request interceptor error:', error);
        return Promise.reject(error);
    }
);

// Добавляем перехватчик ответов (response interceptor)
api.interceptors.response.use(
    (response) => {
        return response;
    },
    (error) => {
        if (error.response && (error.response.status === 401 || error.response.status === 403)) {
            console.error('api.js - Unauthorized or Forbidden access. Redirecting to login or home.');
        }
        return Promise.reject(error);
    }
);

export default api;
```

Рис. 36. React-код сервиса для взаимодействия с API (фрагмент)

Приложение 5

Неполный код компонента профиля пользователя (React)

```
deleteGrade: async (gradeId) => {
    try {
        const token = AuthService.getToken();
        await axios.delete(`${API_BASE_URL}/Grades/${gradeId}`, {
            headers: { Authorization: `Bearer ${token}` }
        });
        return true;
    } catch (error) {
        console.error('Error deleting grade:', error);
        throw error;
    }
},

getAllTeacherSubjectGroupAssignments: async (filters = {}) => {
    try {
        const token = AuthService.getToken();
        const queryString = new URLSearchParams(filters).toString();
        const url = `${API_BASE_URL}/teacher-subject-group-assignments${queryString ? `?${queryString}` : ''}`;
        const response = await axios.get(url, {
            headers: { Authorization: `Bearer ${token}` }
        });
        return response.data;
    } catch (error) {
        console.error('Error fetching teacher-subject-group assignments:', error);
        throw error;
    }
},

addTeacherSubjectGroupAssignment: async (assignmentData) => {
    try {
        const token = AuthService.getToken();
        const response = await axios.post(`${API_BASE_URL}/teacher-subject-group-assignments`, assignmentData, {
            headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' }
        });
        return response.data;
    } catch (error) {
        console.error('Error adding teacher-subject-group assignment:', error);
        throw error;
    }
},
```

Рис. 37. React-код компонента профиля пользователя (фрагмент)

Приложение 6

Неполный код компонента навигации (React)

```
useEffect(() => {
    window.addEventListener('scroll', handleScroll);
    return () => {
        window.removeEventListener('scroll', handleScroll);
    };
}, [handleScroll]);

const logOut = () => {
    AuthService.logout();
    setCurrentUser(undefined);
    navigate('/login');
    setIsMenuOpen(false); // Закрыть меню при выходе
};

const isAdmin = currentUser && AuthService.isAdmin();
const isTeacher = currentUser && AuthService.isTeacher();
const isStudent = currentUser && AuthService.isStudent();

const handleLinkClick = () => {
    setIsMenuOpen(false); // Закрыть меню при клике на ссылку
};

return (
    <nav className={`
        p-4 flex justify-between items-center sticky top-0 z-50 rounded-b-lg
        transition-colors duration-300 ease-in-out
        ${scrolled ? 'bg-blue-800 bg-opacity-90 shadow-xl' : 'bg-transparent shadow-md'}
    `}>
        {/* Логотип и название сайта */}
        <Link to="/" className="flex items-center text-white text-2xl font-bold no-underline transition-colors duration-300 hover:text-blue-100" onClick={handleLinkClick}>
            <img
                src="/favicon.png" // Используем favicon.png для логотипа в Navbar
                alt="Student Performance App Icon"
                className="h-10 w-10 rounded-full object-cover mr-3"
                onError={(e) => { e.target.onerror = null; e.target.src="https://placehold.co/40x40/3B82F6/ffffff?text=APP" }}
            />
            Student Performance App
        </Link>

        {/* Гамбургер-иконка для мобильных - видна только на экранах меньше md */}
        <div className="md:hidden">
            <button onClick={() => setIsMenuOpen(!isMenuOpen)} className="text-white focus:outline-none">
                <svg className="h-8 w-8" fill="none" viewBox="0 0 24 24" stroke="currentColor">
                    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d={isMenuOpen ? "M6 18L18 6M6 6l12 12" : "M4 6h16M4 12h16M4 18h16"} />
                </svg>
            </button>
```

Рис. 38. React-код компонента навигации (фрагмент)

Приложение 7

Неполный код футера сайта (React)

```
{/* Колонка 3: Разделы */}
<div className="flex flex-col items-center text-center md:items-end md:text-right">
  <h5 className="font-semibold text-3xl text-gray-300 mb-6 border-b-2 border-gray-700 pb-3 inline-block relative group">
    Разделы
    <span className="absolute left-0 bottom-0 w-full h-0.5 bg-white scale-x-0 group-hover:scale-x-100 origin-left transition-transform duration-300 ease-out"></span>
  </h5>
  <ul className="space-y-4">
    <li>
      <Link to="/home" className="text-xl hover:text-white transition-colors duration-300 relative group block py-1">
        Главная
        <span className="absolute left-0 bottom-0 w-0 h-0.5 bg-white transition-all duration-300 ease-out group-hover:w-full"></span>
      </Link>
    </li>
    <li>
      <Link to="/profile" className="text-xl hover:text-white transition-colors duration-300 relative group block py-1">
        Профиль
        <span className="absolute left-0 bottom-0 w-0 h-0.5 bg-white transition-all duration-300 ease-out group-hover:w-full"></span>
      </Link>
    </li>
    <li>
      <Link to="/reports" className="text-xl hover:text-white transition-colors duration-300 relative group block py-1">
        Отчеты
        <span className="absolute left-0 bottom-0 w-0 h-0.5 bg-white transition-all duration-300 ease-out group-hover:w-full"></span>
      </Link>
    </li>
    <li>
      <Link to="/login" className="text-xl hover:text-white transition-colors duration-300 relative group block py-1">
        Вход
        <span className="absolute left-0 bottom-0 w-0 h-0.5 bg-white transition-all duration-300 ease-out group-hover:w-full"></span>
      </Link>
    </li>
    <li>
      <Link to="/register" className="text-xl hover:text-white transition-colors duration-300 relative group block py-1">
        Регистрация
        <span className="absolute left-0 bottom-0 w-0 h-0.5 bg-white transition-all duration-300 ease-out group-hover:w-full"></span>
      </Link>
    </li>
  </ul>
</div>
```

Рис.39. React-код футера сайта (фрагмент)

Приложение 8

Структура проекта для разработки веб-сайта

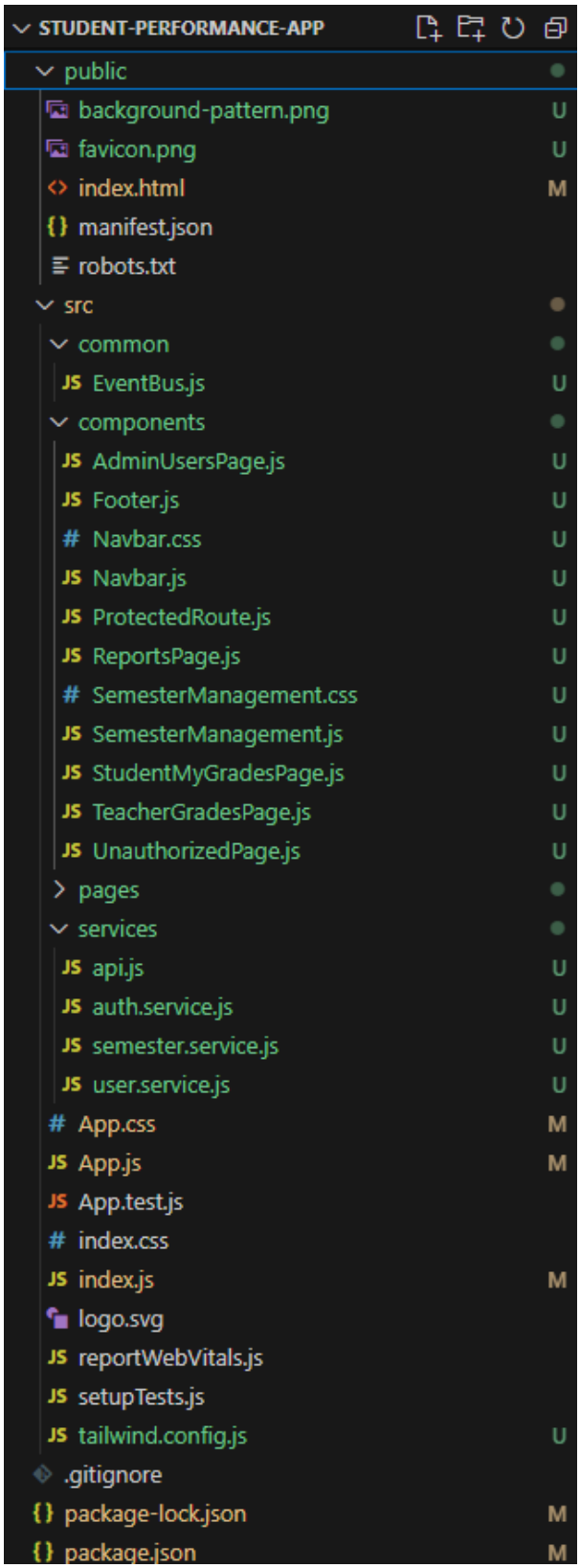

Рис. 40. Структура проекта

Приложение 9

Детальная описательная спецификация прецедентов

1. Прецедент: Авторизация в системе

Краткое описание: Предоставляет пользователю возможность войти в систему, подтвердив свою личность с помощью учетных данных, и получить доступ к функционалу, соответствующему его роли.

- Субъекты: Администратор, Преподаватель, Студент (Все Акторы).
- Предусловия:
  - Система запущена и доступна.
  - Пользователь имеет действующие учетные данные (логин и пароль).
- Основной поток событий:

1. Пользователь открывает страницу входа в систему.
2. Система отображает форму входа.
3. Пользователь вводит свой логин и пароль в соответствующие поля.
4. Пользователь нажимает кнопку "Войти".
5. Система получает учетные данные и отправляет их на сервер для проверки.
6. Сервер проверяет учетные данные пользователя в базе данных.
7. Если учетные данные верны, сервер генерирует JWT-токен и возвращает его клиенту вместе с информацией о пользователе и его роли.
8. Клиентская часть сохраняет токен и информацию о пользователе (например, в локальное хранилище).
9. Система перенаправляет пользователя на главную страницу или страницу, соответствующую его роли.

- Альтернативные потоки:
  - Неверные учетные данные:
    1. На шаге 6: Сервер определяет, что логин или пароль неверны.

2. Сервер возвращает ошибку аутентификации (например, статус 401 Unauthorized).
3. Система отображает сообщение об ошибке (например, "Неверный логин или пароль.").
4. Пользователь может скорректировать данные или отменить вход.

- Аккаунт заблокирован/неактивен:
  1. На шаге 6: Сервер определяет, что аккаунт пользователя заблокирован или неактивен.
  2. Сервер возвращает ошибку (например, статус 403 Forbidden).
  3. Система отображает соответствующее сообщение (например, "Ваша учетная запись заблокирована. Обратитесь к администратору.").

- Постусловия:
  - Если авторизация успешна: Пользователь аутентифицирован, имеет действительный токен, инициирована сессия, и пользователь находится на своей основной странице.
  - Если авторизация неуспешна: Пользователь остается на странице входа, сессия не инициирована.

2. Прецедент: Управление пользователями

- Краткое описание: Предоставляет Администратору возможность добавлять, просматривать, редактировать и удалять учетные записи пользователей (Администраторов, Преподавателей, Студентов), а также назначать им роли.
- Субъекты: Администратор.
- Предусловия:
  - Администратор успешно авторизован в системе.
  - В базе данных существуют необходимые роли (Admin, Teacher, Student).

- Основной поток событий:

1. Администратор переходит в раздел "Управление пользователями".
2. Система отображает список существующих пользователей с возможностью фильтрации и поиска.
3. Добавление пользователя:
    1. Администратор нажимает кнопку "Добавить нового пользователя".
    2. Система отображает форму добавления пользователя.
    3. Администратор вводит данные нового пользователя (логин, пароль, имя, фамилия, email, роль, дополнительные данные для студента/преподавателя).
    4. Администратор нажимает кнопку "Сохранить".
    5. Система валидирует введенные данные.
    6. Система сохраняет новую учетную запись в базе данных.
    7. Система выводит сообщение об успешном добавлении и обновляет список пользователей.
4. Редактирование пользователя:
    1. Администратор выбирает существующего пользователя из списка и нажимает "Редактировать".
    2. Система отображает форму с текущими данными пользователя.
    3. Администратор вносит необходимые изменения в данные пользователя.
    4. Администратор нажимает кнопку "Сохранить изменения".
    5. Система валидирует измененные данные.
    6. Система обновляет данные пользователя в базе данных.
    7. Система выводит сообщение об успешном обновлении и обновляет список пользователей.
5. Удаление пользователя:

1. Администратор выбирает пользователя из списка и нажимает "Удалить".
2. Система запрашивает подтверждение удаления.
3. Администратор подтверждает удаление.
4. Система удаляет учетную запись пользователя из базы данных.
5. Система выводит сообщение об успешном удалении и обновляет список пользователей.

- Альтернативные потоки:
  - Невалидные данные при добавлении/редактировании:
    1. На шаге 5 (Добавление) или 5 (Редактирование): Система обнаруживает невалидные данные (например, слишком короткий пароль, некорректный email).
    2. Система отображает сообщения об ошибках валидации рядом с соответствующими полями формы.
    3. Администратор корректирует данные и повторяет попытку.
  - Логин уже существует:
    1. На шаге 5 (Добавление): Система обнаруживает, что введенный логин уже используется.
    2. Система отображает сообщение об ошибке (например, "Логин уже занят. Выберите другой.").
    3. Администратор изменяет логин и повторяет попытку.
  - Отмена операции:
    1. Администратор в любой момент нажимает кнопку "Отмена" или закрывает форму.
    2. Система прерывает текущую операцию без сохранения изменений.
- Постусловия:

- Если операция успешна: Учетная запись пользователя добавлена/изменена/удалена в соответствии с действиями Администратора.
- Если операция неуспешна: Состояние учетных записей пользователей в системе не изменилось.

3. Прецедент: Управление учебной структурой

- Краткое описание: Позволяет Администратору добавлять, просматривать, редактировать и удалять информацию о студенческих группах, учебных дисциплинах и семестрах в системе.
- Субъекты: Администратор.
- Предусловия:
  - Администратор успешно авторизован в системе.
- Основной поток событий:

1. Администратор переходит в соответствующий раздел управления (например, "Управление группами", "Управление дисциплинами", "Управление семестрами").
2. Система отображает список существующих сущностей (групп, дисциплин, семестров).
3. Добавление сущности:
    1. Администратор нажимает кнопку "Добавить новую [сущность]".
    2. Система отображает форму добавления.
    3. Администратор вводит необходимые данные (например, название и код для группы/дисциплины, название и даты для семестра).
    4. Администратор нажимает кнопку "Сохранить".
    5. Система валидирует введенные данные.
    6. Система сохраняет новую запись в базе данных.
    7. Система выводит сообщение об успешном добавлении и обновляет список сущностей.

4. Редактирование сущности:
    1. Администратор выбирает сущность из списка и нажимает "Редактировать".
    2. Система отображает форму с текущими данными сущности.
    3. Администратор вносит необходимые изменения.
    4. Администратор нажимает кнопку "Сохранить изменения".
    5. Система валидирует измененные данные.
    6. Система обновляет данные сущности в базе данных.
    7. Система выводит сообщение об успешном обновлении и обновляет список сущностей.
5. Удаление сущности:
    1. Администратор выбирает сущность из списка и нажимает "Удалить".
    2. Система запрашивает подтверждение удаления (с учетом возможных связей с другими данными).
    3. Администратор подтверждает удаление.
    4. Система удаляет сущность из базы данных.
    5. Система выводит сообщение об успешном удалении и обновляет список сущностей.

- Альтернативные потоки:
    - Невалидные данные: Система обнаруживает невалидные данные при вводе/редактировании и отображает сообщения об ошибках валидации.
    - Дублирование данных: При добавлении сущности с уже существующим уникальным полем (например, кодом группы), система выдает ошибку.
    - Попытка удаления связанной сущности: При попытке удаления сущности, которая имеет активные связи с другими данными (например, удаление группы, в которой есть студенты), система

предупреждает об этом и/или запрещает удаление до устранения связей.

- Постусловия:
  - Если операция успешна: Информация об учебной структуре (группы, дисциплины, семестры) изменена в соответствии с действиями Администратора.
  - Если операция неуспешна: Состояние данных учебной структуры не изменилось.

4. Прецедент: Назначение преподавателей

- Краткое описание: Предоставляет Администратору возможность назначать преподавателей на определенные учебные дисциплины и/или студенческие группы в рамках конкретного семестра.
- Субъекты: Администратор.
- Предусловия:
  - Администратор успешно авторизован в системе.
  - В системе существуют зарегистрированные преподаватели, дисциплины, группы и семестры.
- Основной поток событий:

1. Администратор переходит в раздел "Назначение преподавателей".
2. Система отображает список существующих назначений.
3. Добавление назначения:
   1. Администратор нажимает кнопку "Добавить новое назначение".
   2. Система отображает форму добавления назначения.
   3. Администратор выбирает из списков:
      - Преподавателя.
      - Дисциплину.
      - Группу (опционально, если дисциплина преподается конкретной группе).
      - Семестр.

4. Администратор нажимает кнопку "Сохранить".
5. Система валидирует выбранные параметры.
6. Система создает новое назначение в базе данных.
7. Система выводит сообщение об успешном добавлении и обновляет список назначений.

4. Редактирование назначения: (Если применимо, например, изменение группы или семестра для существующего назначения)

1. Администратор выбирает существующее назначение и нажимает "Редактировать".
2. Система отображает форму с текущими параметрами назначения.
3. Администратор вносит необходимые изменения.
4. Администратор нажимает кнопку "Сохранить изменения".
5. Система валидирует измененные данные и обновляет назначение.
6. Система выводит сообщение об успешном обновлении.

5. Удаление назначения:

1. Администратор выбирает назначение из списка и нажимает "Удалить".
2. Система запрашивает подтверждение.
3. Администратор подтверждает.
4. Система удаляет назначение из базы данных.
5. Система выводит сообщение об успешном удалении.

- Альтернативные потоки:
  - Некорректные параметры: Система обнаруживает невалидные комбинации (например, попытка назначить несуществующие сущности) и выдает ошибку валидации.
  - Дублирующее назначение: Система предотвращает создание назначения, которое уже существует (например, тот же преподаватель, дисциплина, группа и семестр).

    - Назначение имеет связанные оценки: Если назначение уже имеет связанные оценки, система предупреждает Администратора и/или запрещает удаление до тех пор, пока оценки не будут перенесены или удалены.
- Постусловия:
    - Если операция успешна: Преподаватель назначен/переназначен/удален из дисциплины/группы в указанный семестр.
    - Если операция неуспешна: Состояние назначений преподавателей не изменилось.

5. Прецедент: Ввод/редактирование оценок

- Краткое описание: Позволяет Преподавателю вносить новые оценки студентам по своим дисциплинам и корректировать ранее выставленные оценки.
- Субъекты: Преподаватель.
- Предусловия:
    - Преподаватель успешно авторизован в системе.
    - Преподавателю назначены дисциплины и группы в текущем/активном семестре.
    - В системе зарегистрированы студенты в назначенных группах.
- Основной поток событий:

1. Преподаватель переходит в раздел "Мои дисциплины" или "Ввод оценок".
2. Система отображает список назначенных преподавателю дисциплин и групп.
3. Преподаватель выбирает учебную дисциплину, группу и семестр.
4. Система отображает список студентов выбранной группы для данной дисциплины и текущие оценки (если есть).
5. Ввод новой оценки:

1. Преподаватель выбирает студента и поле для ввода новой оценки (например, за контрольную работу, экзамен).
2. Преподаватель вводит значение оценки (например, числовое, текстовое), тип контроля, дату получения.
3. Преподаватель нажимает кнопку "Сохранить".
4. Система валидирует введенные данные (например, диапазон оценки).
5. Система сохраняет новую оценку в базе данных.
6. Система выводит сообщение об успешном сохранении.

6. Редактирование существующей оценки:
    1. Преподаватель выбирает существующую оценку для студента и нажимает "Редактировать".
    2. Система отображает форму с текущими данными оценки.
    3. Преподаватель изменяет значение оценки, тип контроля, дату или примечания.
    4. Преподаватель нажимает кнопку "Сохранить изменения".
    5. Система валидирует измененные данные.
    6. Система обновляет данные оценки в базе данных.
    7. Система выводит сообщение об успешном обновлении.

- Альтернативные потоки:
    - Невалидные данные: Система обнаруживает, что введенная оценка вне допустимого диапазона или имеет некорректный формат, и отображает сообщение об ошибке валидации.
    - Невозможность сохранения: Если произошла ошибка при сохранении в базу данных (например, проблемы с соединением), система выводит соответствующее сообщение об ошибке.
    - Отмена операции: Преподаватель может отменить ввод/редактирование, не сохраняя изменения.
- Постусловия:

- Если операция успешна: Оценка студента по выбранной дисциплине добавлена или изменена в базе данных.
- Если операция неуспешна: Состояние оценок в системе не изменилось.

6. Прецедент: Просмотр успеваемости студента

- Краткое описание: Позволяет Студенту получить и отобразить подробную информацию о своих академических результатах (оценках, посещаемости) по всем дисциплинам за различные учебные периоды.
- Субъекты: Студент.
- Предусловия:
  - Студент успешно авторизован в системе.
  - В базе данных существуют оценки для данного студента.
- Основной поток событий:

1. Студент входит в свой личный кабинет или переходит в раздел "Моя успеваемость".
2. Система автоматически определяет текущего студента.
3. Система запрашивает из базы данных все оценки и данные о посещаемости, относящиеся к данному студенту.
4. Система отображает информацию об успеваемости студента в удобном для восприятия формате (например, таблица с оценками по дисциплинам и семестрам, сводная статистика).
5. Студент может использовать фильтры (по семестру, дисциплине) для более детального просмотра.
6. Система обновляет отображаемую информацию в соответствии с фильтрами.

- Альтернативные потоки:
  - Отсутствие оценок: Если для студента в базе данных не найдено оценок, система отображает сообщение "Оценки пока отсутствуют" или "Нет данных для отображения".

    - Проблемы с загрузкой данных: Если произошла ошибка при запросе данных с сервера, система выводит сообщение об ошибке.
- Постусловия:
    - Если успешно: Студент просматривает актуальную информацию о своей успеваемости.
    - Если неуспешно: Студент получает сообщение об отсутствии данных или ошибке.

Приложение 9.7. Прецедент: Просмотр успеваемости группы

- Краткое описание: Предоставляет Преподавателю или Администратору возможность получить и отобразить сводную информацию об успеваемости всех студентов в выбранной группе.
- Субъекты: Преподаватель, Администратор.
- Предусловия:
    - Преподаватель или Администратор успешно авторизован в системе.
    - В системе существуют зарегистрированные группы и студенты, а также оценки.
- Основной поток событий:

1. Пользователь (Преподаватель или Администратор) переходит в раздел "Успеваемость групп" или "Отчеты по группам".
2. Система отображает список доступных групп.
3. Пользователь выбирает учебную группу и, возможно, конкретную дисциплину или семестр для просмотра.
4. Система запрашивает из базы данных оценки всех студентов выбранной группы по выбранным критериям.
5. Система агрегирует данные (например, рассчитывает средний балл по группе, количество неуспевающих).
6. Система отображает сводную информацию об успеваемости группы в табличном или графическом виде.

7. Пользователь может использовать дополнительные фильтры или опции сортировки для анализа данных.

- Альтернативные потоки:
  - Отсутствие данных: Если для выбранной группы/дисциплины/семестра нет оценок, система отображает сообщение об отсутствии данных.
  - Пустая группа: Если в выбранной группе нет студентов, система информирует об этом.
  - Проблемы с загрузкой/агрегацией данных: Система выводит сообщение об ошибке, если не удалось загрузить или обработать данные.
- Постусловия:
  - Если успешно: Пользователь просматривает актуальную сводную информацию об успеваемости выбранной группы.
  - Если неуспешно: Пользователь получает сообщение об отсутствии данных или ошибке.

8. Прецедент: Проведение анализа успеваемости

- Краткое описание: Позволяет Преподавателю или Администратору инициировать различные процедуры анализа данных успеваемости (расчет статистики, анализ динамики, выявление тенденций).
- Субъекты: Преподаватель, Администратор.
- Предусловия:
  - Пользователь успешно авторизован в системе.
  - В базе данных достаточно данных (оценок, посещаемости) для проведения анализа.
- Основной поток событий:

1. Пользователь переходит в раздел "Аналитика" или "Отчеты".

2. Система предлагает выбор доступных видов анализа (например, "Средний балл по дисциплинам", "Динамика успеваемости студента", "Распределение оценок").
3. Пользователь выбирает тип анализа и задает необходимые параметры (например, конкретный студент, группа, дисциплина, временной период).
4. Пользователь нажимает кнопку "Запустить анализ".
5. Система собирает необходимые исходные данные из базы данных.
6. Система выполняет выбранный алгоритм анализа данных.
7. Система подготавливает результаты анализа для отображения.

- Альтернативные потоки:
  - Недостаточно данных: Если для выбранных параметров недостаточно данных для проведения анализа, система выводит сообщение об этом (например, "Недостаточно оценок для построения динамики.").
  - Некорректные параметры: Если параметры анализа невалидны, система выдает ошибку.
  - Ошибка при выполнении анализа: Если в процессе анализа произошла внутренняя ошибка, система выводит сообщение об этом.
- Постусловия:
  - Если успешно: Результаты анализа данных готовы к просмотру или формированию отчета (см. прецедент "Просмотр результатов анализа").
  - Если неуспешно: Анализ не выполнен, отображается сообщение об ошибке или отсутствии данных.

9. Прецедент: Просмотр результатов анализа

- Краткое описание: Предоставляет Преподавателю или Администратору возможность отобразить результаты проведенного анализа

успеваемости в удобном для восприятия формате (таблицы, графики, диаграммы).

- Субъекты: Преподаватель, Администратор.
- Предусловия:
  - Пользователь успешно авторизован в системе.
  - Анализ успеваемости (см. прецедент "Проведение анализа успеваемости") был успешно выполнен и его результаты доступны.
- Основной поток событий:

1. После выполнения анализа (или при переходе в раздел с результатами анализа) система отображает подготовленные данные.
2. Система визуализирует результаты в виде таблиц, графиков (линейные, столбчатые) или круговых диаграмм, в зависимости от типа анализа.
3. Пользователь может взаимодействовать с визуализацией (например, наводить курсор для получения деталей, изменять масштаб).

- Альтернативные потоки:
  - Отсутствие результатов: Если результаты анализа недоступны (например, анализ не был запущен или завершился с ошибкой), система отображает сообщение об отсутствии данных.
  - Ошибка визуализации: Если произошла ошибка при построении графиков/таблиц, система выводит сообщение об этом.
- Постусловия:
  - Если успешно: Пользователь просматривает результаты анализа.
  - Если неуспешно: Результаты анализа не отображаются, выводится сообщение об ошибке.

10. Прецедент: Формирование отчетов

- Краткое описание: Позволяет Администратору генерировать структурированные отчеты по успеваемости на основе заданных параметров с возможностью экспорта в различных форматах.

- Субъекты: Администратор.
- Предусловия:
    - Администратор успешно авторизован в системе.
    - В базе данных существуют данные для формирования отчета.
- Основной поток событий:

1. Администратор переходит в раздел "Отчеты".
2. Система отображает список доступных шаблонов отчетов (например, "Сводный отчет по успеваемости группы", "Отчет по индивидуальной успеваемости студента").
3. Администратор выбирает тип отчета и задает необходимые параметры (например, группа, семестр, студент, диапазон дат).
4. Администратор выбирает формат экспорта (например, PDF, Excel).
5. Администратор нажимает кнопку "Сформировать отчет".
6. Система собирает и обрабатывает данные согласно выбранному типу отчета и параметрам.
7. Система генерирует отчет в выбранном формате.
8. Система предлагает Администратору скачать сформированный файл отчета.

- Альтернативные потоки:
    - Недостаточно данных: Если для выбранных параметров нет данных для формирования отчета, система выводит сообщение об этом.
    - Ошибка генерации: Если произошла ошибка в процессе формирования файла отчета (например, проблемы с шаблоном, нехватка ресурсов), система выводит соответствующее сообщение.
    - Невалидные параметры: Если параметры для отчета некорректны, система выдает ошибку валидации.
- Постусловия:
    - Если успешно: Отчет сформирован и доступен для скачивания.

- Если неуспешно: Отчет не сформирован, выводится сообщение об ошибке.

Приложение 10

Детализация сущностей ER-диаграммы

1. Сущности и атрибуты

Пользователи (Users)

- UserId (INT, PK) - Идентификатор пользователя.
- Username (NVARCHAR(50), Unique) - Имя пользователя (логин).
- PasswordHash (NVARCHAR(255)) - Хешированный пароль.
- Email (NVARCHAR(100), Unique) - Электронная почта.
- FirstName (NVARCHAR(50)) - Имя.
- LastName (NVARCHAR(50)) - Фамилия.
- Patronymic (NVARCHAR(50), Nullable) - Отчество.
- UserType (NVARCHAR(20)) - Тип пользователя ('Admin', 'Teacher', 'Student').
- CreatedAt (DATETIME) - Дата создания записи.
- UpdatedAt (DATETIME) - Дата последнего обновления.

Студенты (Students)

- StudentId (INT, PK) - Идентификатор студента.
- UserId (INT, FK to Users) - Ссылка на пользователя.
- GroupId (INT, FK to Groups) - Ссылка на группу.
- DateOfBirth (DATE) - Дата рождения.

Преподаватели (Teachers)

- TeacherId (INT, PK) - Идентификатор преподавателя.
- UserId (INT, FK to Users) - Ссылка на пользователя.
- HireDate (DATE) - Дата найма.
- Department (NVARCHAR(100)) - Отдел.
- Position (NVARCHAR(100)) - Должность.

Группы (Groups)

- GroupId (INT, PK) - Идентификатор группы.
- GroupName (NVARCHAR(50), Unique) - Название группы.

- Year (INT) - Год создания группы (год набора).

Предметы (Subjects)

- SubjectId (INT, PK) - Идентификатор предмета.
- Name (NVARCHAR(100), Unique) - Название предмета.
- Description (NVARCHAR(MAX), Nullable) - Описание предмета.

Семестры (Semesters)

- SemesterId (INT, PK) - Идентификатор семестра.
- Name (NVARCHAR(50)) - Название семестра (напр. "Осень 2023").
- StartDate (DATE) - Дата начала семестра.
- EndDate (DATE) - Дата окончания семестра.

Назначения: Преподаватель-предмет-группа (TeacherSubjectGroupAssignments)

- TeacherSubjectGroupAssignmentId (INT, PK) - Идентификатор назначения.
- TeacherId (INT, FK to Teachers) - Ссылка на преподавателя.
- SubjectId (INT, FK to Subjects) - Ссылка на предмет.
- GroupId (INT, FK to Groups) - Ссылка на группу.
- SemesterId (INT, FK to Semesters) - Ссылка на семестр.

Оценки (Grades)

- GradeId (INT, PK) - Идентификатор оценки.
- StudentId (INT, FK to Students) - Ссылка на студента.
- TeacherSubjectGroupAssignmentId (INT, FK to TeacherSubjectGroupAssignments) - Ссылка на назначение (преподаватель-предмет-группа).
- Value (NVARCHAR(10)) - Значение оценки (может быть "5", "Зачет", "Н/А").
- DateReceived (DATETIME) - Дата получения оценки.
- ControlType (NVARCHAR(50)) - Тип контроля ("Экзамен", "Зачет", "Курсовая работа").

- Status (NVARCHAR(50), Nullable) - Статус оценки ("Получено", "Пересдача").
- Notes (NVARCHAR(MAX), Nullable) - Дополнительные заметки.

2. Взаимосвязи между сущностями

- Users (1) --- (1) Students: Каждый студент является пользователем.
- Users (1) --- (1) Teachers: Каждый преподаватель является пользователем.
- Groups (1) --- (*) Students: Одна группа может иметь много студентов.
- Subjects (1) --- (*) TeacherSubjectGroupAssignments: Один предмет может быть назначен во многих связках преподаватель-группа.
- Semesters (1) --- (*) TeacherSubjectGroupAssignments: Один семестр может быть связан со многими назначениями.
- Teachers (1) --- (*) TeacherSubjectGroupAssignments: Один преподаватель может иметь много назначений.
- Groups (1) --- (*) TeacherSubjectGroupAssignments: Одна группа может быть связана со многими назначениями.
- TeacherSubjectGroupAssignments (1) --- (*) Grades: Одно назначение (преподаватель-предмет-группа-семестр) может иметь много оценок (для разных студентов).
- Students (1) --- (*) Grades: Один студент может иметь много оценок.

Приложение 11

Мобильные версии страниц гостевой части

1. Главная страница в мобильной версии

На данном скриншоте показан вид главной страницы приложения при просмотре на мобильном устройстве. Элементы интерфейса (навигационное меню, баннер, основные информационные блоки) адаптированы для вертикальной прокрутки, обеспечивая оптимальное использование пространства экрана смартфона.

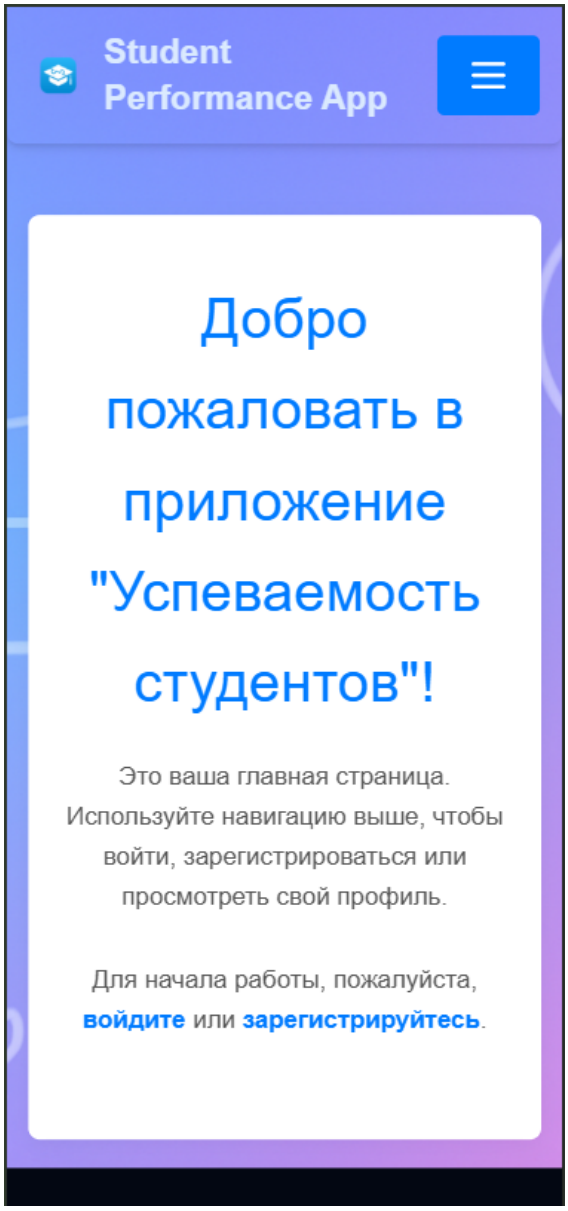

Рис. 41. Главная страница в мобильной версии

2. Раздел "Навигация" для Администратора в мобильной версии

Скриншот демонстрирует раскрытое навигационное меню для пользователя с ролью "Администратор" на мобильном устройстве. Меню, содержащее ссылки на административные разделы, оптимизировано для удобного взаимодействия через касания.

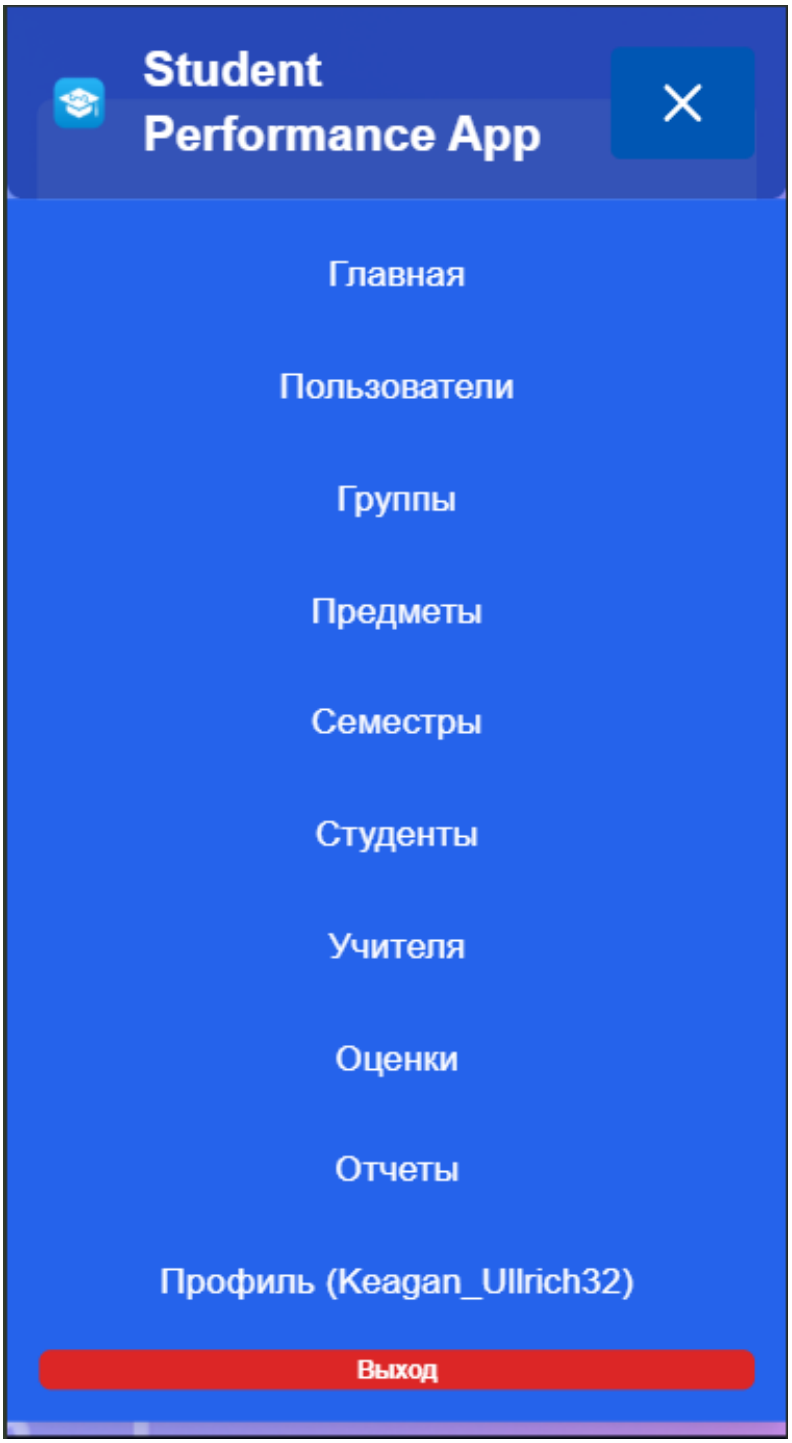

Рис. 42. навигационное меню на мобильном устройстве

3. Футер сайта в мобильной версии

На этом скриншоте представлен вид нижней части сайта (футера) на мобильном устройстве. Элементы футера, такие как контактная информация, ссылки на социальные сети или дополнительные разделы, также адаптированы для небольших экранов.

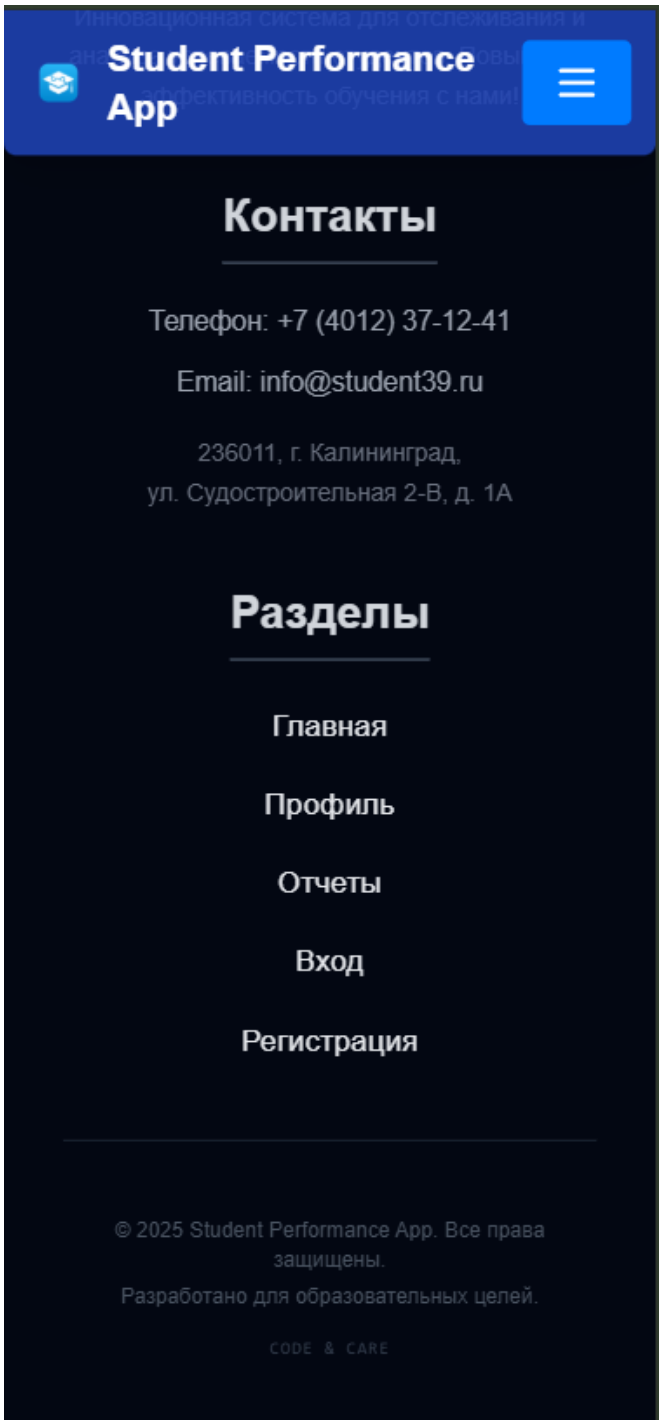

Рис. 43. Футер сайта в мобильной версии

3. Страница авторизации в мобильной версии

Скриншот показывает страницу входа в систему, адаптированную для мобильного устройства. Форма ввода логина и пароля, а также кнопки и ссылки, расположены таким образом, чтобы обеспечить легкое и точное взаимодействие.

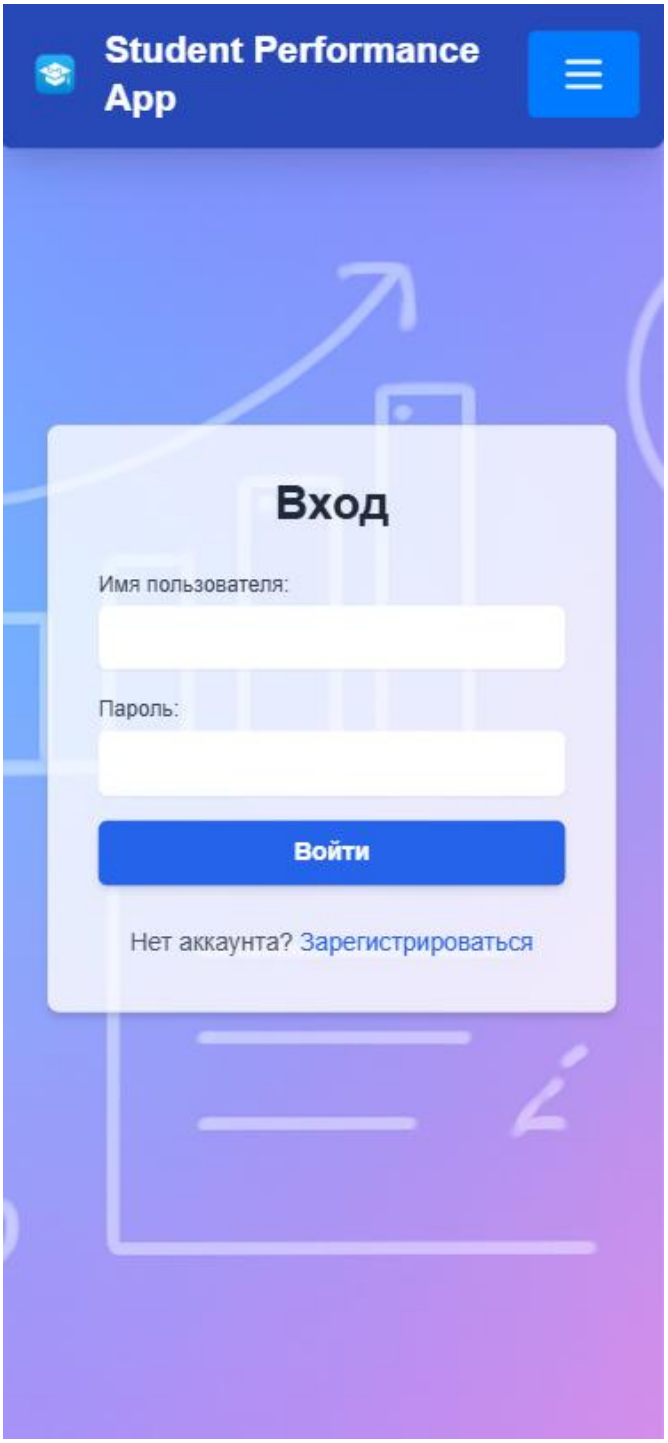

Рис. 44. Страница авторизации

5. Страница регистрации в мобильной версии

На данном скриншоте отображена страница регистрации нового пользователя при просмотре на мобильном устройстве. Поля формы регистрации и элементы управления корректно масштабируются и расположены для удобного заполнения.

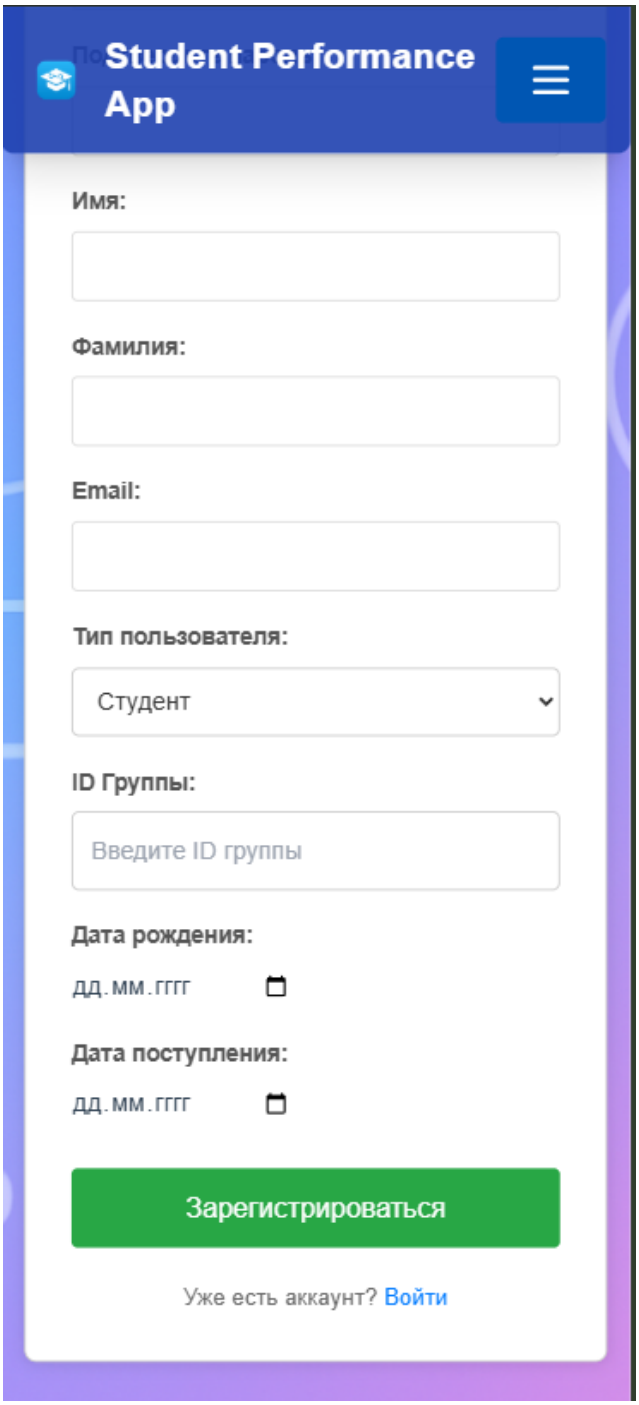

Рис. 45. Страница регистрации

Приложение 12

Детальное описание клиентских компонентов и адаптивного дизайна

1. Ключевые клиентские компоненты и их функциональность

Разработка клиентской части базировалась на модульной, компонентной структуре, обеспечиваемой React.js. Основные задачи включали обеспечение легкого доступа к функциям, интуитивно понятной навигации, высокой скорости работы и адаптивного дизайна.

Для реализации использовались: React.js, JavaScript (ES6+), HTML, Tailwind CSS, react-router-dom, axios, date-fns, lucide-react.

2.Навигация (Navbar, Footer)

Navbar (верхнее меню) и Footer (нижний колонтитул): Эти компоненты доступны на всех страницах приложения. Они обеспечивают легкий доступ к основным разделам, функциям аутентификации (вход, регистрация) и ссылкам профиля пользователя. Дизайн выполнен таким образом, чтобы быть функциональным и эстетически привлекательным на любых размерах экрана.

3. Защита маршрутов (ProtectedRoute)

ProtectedRoute: Это пользовательский компонент маршрутизации, разработанный для обеспечения безопасности и контроля доступа к страницам приложения. Он проверяет статус аутентификации пользователя и его роль перед рендерингом защищенного контента. Если пользователь не авторизован или не имеет необходимых прав, он перенаправляется на страницу входа или страницу с сообщением о запрете доступа. Это обеспечивает гибкую и надежную ролевую модель доступа на клиентской стороне.

4. Вспомогательные UI-компоненты

Для повышения переиспользуемости кода, унификации внешнего вида и улучшения пользовательского опыта были разработаны следующие вспомогательные UI-компоненты:

- **CustomAlert**: Переиспользуемый компонент для отображения уведомлений различного типа (успех, ошибка, информация, предупреждение). Позволяет выводить сообщения пользователю в унифицированном стиле, обеспечивая при этом гибкую настройку (например, автоскрытие, возможность закрытия вручную).
- CustomSpinner: Компонент индикатора загрузки. Используется для отображения состояния ожидания во время выполнения асинхронных операций (например, загрузка данных с сервера, сохранение изменений), улучшая обратную связь для пользователя и предотвращая ощущение "зависания" приложения.
- CustomModal: Унифицированный компонент для создания модальных окон. Применяется для форм добавления, редактирования и удаления данных, а также для подтверждения действий. Обеспечивает стандартизированный внешний вид и поведение модальных окон по всему приложению.

5. Реализация адаптивного дизайна

Для обеспечения корректного отображения и удобства использования сайта на различных устройствах и в разных браузерах были применены следующие подходы и техники:

- Использование утилит адаптивности Tailwind CSS: Широкое применение классов-префиксов Tailwind CSS (например, sm:, md:, lg:, xl:) позволило тонко адаптировать макет, отступы, типографику, размеры элементов и их видимость на различных точках останова (breakpoints). Это исключило необходимость написания большого объема пользовательского медиа-запросов.
- Гибкие элементы: Активное использование CSS Flexbox (flex) и Grid (grid) для построения отзывчивых макетов. Компоненты и контейнеры используют относительные единицы измерения (%, vw/vh) и классы Tailwind (w-full, max-w-full, h-auto для изображений), что обеспечивает

плавное изменение их размеров в зависимости от доступного пространства и предотвращает появление горизонтальной прокрутки.

- Тестирование: Проведено всестороннее тестирование приложения в различных современных веб-браузерах (Google Chrome, Mozilla Firefox, Safari, Microsoft Edge) и на эмуляторах мобильных устройств для проверки совместимости, корректного отображения и пользовательского опыта на разных платформах.
- Оптимизация: Выполнена общая оптимизация клиентского кода (например, ленивая загрузка компонентов, мемоизация) и оптимизация изображений (если таковые использовались) для ускорения загрузки страниц и повышения общей производительности приложения.

В совокупности эти подходы обеспечили создание современного, функционального и удобного веб-ресурса, который соответствует всем поставленным задачам и требованиям к пользовательскому опыту и адаптивности, обеспечивая легкий доступ к информации и интуитивно понятную навигацию.